

Design and Implementation of an Interactive IOT Network Anomaly Detection System Using Deep
Autoencoders

By

By Mohamed Ibrahim

Submitted to

The University of Roehampton

In partial fulfilment of the requirements

for the degree of

BACHELOR OF ENGINEERING IN software engineering

Declaration

I hereby certify that this report constitutes my own work, that where the language of others is used, quotation marks so indicate, and that appropriate credit is given where I have used the language, ideas, expressions, or writings of others.

I declare that this report describes the original work that has not been previously presented for the award of any other degree of any other institution.

Mohamed Ibrahim

Date: 11/05/2026

A handwritten signature in black ink. The word "Ibrahim" is written in a cursive style. A large, thick, black circular stroke is drawn over the "I" and the beginning of the "Ibrahim", partially obscuring it. The signature is written on a white background.

Acknowledgements

It is my pleasure to acknowledge the contribution made by my first supervisor Saeed Fakhreldin Saeed and my second supervisor Lisa Haskell towards the completion of this project. I also thank the University of Roehampton for offering me the facilities and assistance necessary to undertake this project. Lastly, I want to thank my family members and peers for motivating me.

Abstract

The current project developed and deployed an IoT traffic anomaly detection system, based on deep autoencoders, which were expected to detect unusual network traffic behaviours and represent the findings graphically. The implementation involved preprocessing IoT traffic datasets, training a deep autoencoder, calculating anomaly scores and classifying records as normal or anomalies using the Python programming language, TensorFlow / Keras, Scikit-learn and Streamlit libraries. Additionally, an Isolation Forest algorithm was applied as a comparative model. To evaluate the performance of both models, we utilized precision, recall, F1 score and PR-AUC instead of the accuracy metric due to the high imbalance of classes during anomaly detection. The analysis demonstrated that the prototype was able to perform the necessary actions. However, the performance highly relied on the proper choice of the threshold value and the quality of the input data.

Table of Contents

Table of Contents

Acknowledgements.....	3
Abstract.....	3
Table of Contents.....	4
Table of Contents.....	4
Problem Statement and/or Research question	8
Research question.....	8
Aims	9
Objectives	9
Section 2. Background and Context.....	11
Background	11
Summary of relevant published work (Literature review).....	12
Problem landscape: why IoT anomaly detection is difficult.	12
Section 3. Tools and Methods.....	16
Rational for technical tools selected.....	16
TensorFlow / Keras vs PyTorch: Justifiable Choice	16
Streamlit vs Flask: Justification for Rapid Prototype Dashboard Delivery.....	18
Rationale for research methods used	19
Research paradigm.....	19
Dataset selection: IoT-23 compared with other benchmarks	19
Evaluation Metrics: F1-Score and PR-AUC vs Accuracy	20
Baseline Comparator: Isolation Forest	20
Reproducibility Strategy	21
Rationale for Project Management Tools and Product Lifecycle used	21
Alignment between Agile principles and the ML Development Cycle.....	21
Sprint structure for Iterative Hyperparameter Tuning	22
Waterfall: Why it falls for Deep Learning Projects.....	24
Risk Mitigation Through Incremental Delivery	25
Legal, ethical and professional issues.....	26
GDPR Compliance and Network Traffic Data	26
The Ethics of False Positives in Critical IoT Environments	26
IoT-23 Dataset: Privacy Risk Mitigation.....	27
Risk Register	28
Gantt Chart	32

Section 4. Requirements/Specifications	33
System Overview and Architecture	33
Requirements and Stakeholders.....	33
Functional Requirements.....	33
Figure 4.2 Use case diagram for the IoT Anomaly Detection System (Network Administrator actor)	34
Non-Functional Requirements.....	34
Section 5. Solution design.....	36
5.1 Design Overview.....	37
5.2 System Architecture.....	38
Layer 1.....	39
Layer 2.....	39
Layer 3.....	39
Layer 4.....	39
5.3 Data pipeline design.....	40
.....	40
Loading of raw data.....	40
Feature selection and cleansing	40
Train / validation / test split	40
Normalisation	41
Autoencoder training	41
Threshold calibration	41
Inference and MSE scoring.....	41
Evaluation and dashboard generation	41
5.4 Threshold Design.....	42
5.5 Design Decisions.....	44
5.6 System interaction (UML Sequence Diagram)	46
Section 6. Solution implementation	47
This chart explains how the concept of reconstruction error was utilized to distinguish normal network traffic from the abnormal one. The line of threshold signifies that any data with a reconstruction error higher than the threshold can be considered an anomaly. It works since any kind of malicious activity would pose a greater challenge to the autoencoder's accurate reconstruction.....	51
Streamlit Dashboard	51
Streamlit allows creating an interactive dashboard for analyzing results of anomaly detection along with important metrics.	51
Section 7. Solution testing/ evaluation.....	52

The model performance was tested using precision, recall, F1-score and prc-auc. Accuracy was not considered for the main criterion for evaluating the model since the anomaly dataset could be unbalanced. Is it possible to have an accurate model that predicts most instances to be normal while ignoring crucial attacks. 52

Confusion matrix 53

The confusion matrix is used to compare the prediction made by the algorithm against the actual labels from the labeled dataset. It helps to see the impact of false positives and false negatives on the performance of the algorithm, as in the case of intrusion detection, the consequences of missing an attack are greater than flagging normal data. 53

Precision recall curve 53

Reconsruction error distribution 54

The distribution of the reconstruction errors will reveal the distribution of benign and malicious records according to the reconstruction errors. It is expected that the benign data will form clusters for low reconstruction errors, whereas malicious data is seen in high reconstruction errors..... 54

Autoencoder vs Isolation Forest..... 54

Autoencoder is tested against the Isolation Forest model in order to determine whether the suggested deep learning techniques outperform the anomaly detection system. These findings suggest that the Autoencoder had greater precision, F1-score and PR-AUC..... 54

Section 8. Conclusion 55

Discussion..... 55

Section 9. References..... 56

Appendices..... 59

Name: Sarah Lim 60

 Previous Experience:..... 60

 Frustrations 60

 Expectations:..... 60

 Background Description..... 60

 Demographics 60

 Goals 60

 Motivations 60

 • Avoid privacy invasion 60

 • Avoid devices being used in botnet attacks..... 60

 • Peace of mind 60

Name: Amir Hassan..... 61

 Previous Experience:..... 61

 Frustrations 61

 Expectations:..... 61

 Background Description..... 61

Demographics	61
Goals	61
Motivations	61
• High grade and strong final report	61
• Build a portfolio quality security /ML project.....	61
• Learn reproducible ML experimentation habits	61

Section 1. Introduction

The IoT has moved from being a novel technology solution to a critical infrastructure element, where there are an estimated 21.1 billion globally connected devices in 2025, which will increase to over thirty-nine billion by 2030 [20]. Such saturation and heterogeneity of connected devices, spanning healthcare applications, industrial automation, and smart utility solutions, have completely destabilised current network security models. IoT environments cannot be compared to other enterprise settings due to their extreme heterogeneity, inconsistencies in protocol standards, and complete lack of embedded security mechanisms.

Such structural differences translate into real world implications that are hard to underestimate. Several breaches in IoT environments have caused disruptions in hospitals, manipulated industrial control systems, and exposed millions of users to massive data harvesting [1], [3]. With a continuous proliferation of IoT deployments in key areas of interest, the problem of insufficient anomaly detection solutions is more relevant than ever before. This is where our project finds its place. Modern studies have gradually gravitated towards deep autoencoders as a feasible solution, harnessing the concept that an anomaly in network traffic, due to its dissimilarity to learnt normal behaviour, causes increased reconstruction errors during inference [2], [13], [16]. Reconstruction error is especially effective in heterogeneous IoT settings, where varying traffic patterns across different devices make labeled data-based techniques unsuitable. This context provides the backdrop for our current study.

Problem Statement and/or Research question

The first approach, signature-based IDS, uses the matching of traffic against static pattern libraries, which makes it inherently incapable of detecting zero-day attacks, i.e., new attack types that have not been classified yet. This includes, for example, the development of new botnet malware that can evade the signature-based IDS because there is no known pattern for comparison [1], [3]. The second approach, supervised machine learning classifiers, circumvents this limitation while creating a dependence on high-quality datasets of manually annotated samples for training purposes. It is impossible to create such training sets in IoT networks, which involve traffic from different devices, operating systems, firmware versions, and even network topologies [3], [4]. Moreover, the use of either approach requires extensive knowledge of machine learning techniques that cannot be found among administrators and operators of IoT networks.

This demonstrates an evident operational gap that cannot be bypassed without developing a new type of IDS for IoT networks. The current models are unable to detect new threats while being too complicated to implement in the operational context.

This project will fill this operational gap by developing an interactive network anomaly detector based on deep autoencoder architectures trained using the principles of unsupervised learning. These neural networks can compress traffic data into latent variables, thereby capturing the typical features of normal traffic. They can detect deviations from this normality through statistical analysis and indicate a potential cyber threat [2], [16]. The creation of an interactive interface based on this solution solves both identified problems and creates a tangible artifact.

Research question

In what way is it possible for an unsupervised deep autoencoder neural network structure to successfully detect any anomaly in diverse IoT traffic patterns, considering that its performance will be evaluated in terms of the F1-Score and Precision-Recall AUC while ensuring a very low level of false positive detections using the IoT-23 dataset?

Aims

The objective of this research is to construct and critically analyse an unsupervised IoT anomaly detection solution which utilizes the engineering of a deep autoencoder model through reconstruction error for modelling normal traffic behaviour, where the performance is measured based on the Isolation Forest approach through F1-Score and PR-AUC using the IoT-23 dataset, while at the same time providing an interactive Streamlit dashboard for use by non-expert users.

Objectives

Objective 1 – Synthesis

Critical synthesis of extant literature regarding network anomaly detection in IoT systems, unsupervised learning paradigms, and deep autoencoders, highlighting theoretical underpinnings, methodological constraints, and gaps evidenced to support the development of the proposed system.

Objective 2 – Engineering

Engineering a reproducible data pipeline for the IoT-23 data, incorporating data ingestion, feature extraction, normalization, and training-test split components robust to heterogeneity of the traffic features within real-world IoT network conditions.

Objective 3 – Development

Development of a deep autoencoder neural network structure to learn low – dimensional representations of normal traffic of IoT networks in an unsupervised fashion, optimizing the reconstruction error threshold parameter to achieve maximum anomaly classification capacity about novel and unlabelled attack patterns.

Objective 4 – Design and Implementation

Design and implementation of an interactive and operable anomaly detection dashboard with Streamlit that will enable non – expert in machine learning personnel to view the distribution of reconstruction errors, investigate anomalies flagged by the model, and interact with other output features.

Objective 5 – Critical Evaluation

Critical evaluation of the performance of the deep autoencoder against a state-of-the-art anomaly detection algorithm known as Isolation Forest with F1-Score and Precision-Recall Area under the Curve (PR-AUC) metrics as evaluation criteria.

Section 2. Background and Context

Background

2.1 IoT security in 2025

The integration of IoT devices across infrastructure sectors has resulted in the creation of a security attack surface unlike anything seen before. Unlike standard computing endpoints, the processing requirements of IoT devices are extremely constrained, using microcontroller-class processors clocked in megahertz and featuring megabyte- or even kilobyte-level memory allocations, compared to gigahertz-class performance and gigabytes of memory allocated in general purpose computing platforms. Such architectural constraints preclude the possibility of model inference on the actual device itself, since computationally intensive supervised learning algorithms necessitate large scale memory allocation, multi pass training, and floating-point arithmetic calculations, all beyond the capacity of the constrained environment of the device. As a result, the architecture of a potential security solution based on machine learning inference is inherently incompatible with IoT endpoints unless there is some sort of significant computation offload from the node itself.

2.2 Edge-Cloud Architecture

The Edge-Cloud architecture is currently the primary means of overcoming the processing constraints of the Internet of Things while simultaneously providing intelligence for security applications. This system divides the network into three complementary tiers, namely the device tier, which comprises the IoT devices themselves; the edge tier, which encompasses compute nodes local to the environment in question that perform aggregation and low latency processing tasks; and the cloud management tier, which handles the global monitoring and coordination of security policies alongside model training and retraining. Such architecture allows reducing reliance on cloud-based compute, eliminating transmission delays, and allowing decentralisation of security enforcement procedures, which is critical when dealing with large scale, distributed IoT network environments. Nevertheless, Edge-Cloud architecture opens the possibility of attacks on communication channels between tiers, necessitating reliable and efficient anomaly detection at the edge.

2.3 Flow based vs Packet based traffic Analysis.

Network anomaly detection systems are classified based on their underlying analysis techniques as being either packet based or flow based. The former refers to methods called Deep Packet Inspection (DPI), whereby each individual packet of traffic is captured and inspected through header analysis and or payload inspection. Although theoretically comprehensive, packet-based analysis is prohibitively computationally costly and is not applicable to multi-gigabit networks. And it becomes entirely impossible in scenarios with encrypted payload. Flow based analysis, on the other hand, deals with statistical descriptions of network flows, which include metadata such as source/destination IP address, port number, byte count, packet count, and flow duration. Such an approach

drastically cuts down the amount of information to be processed in comparison with DPI, which facilitates real time processing at scale, and its agnostic to encryption, allowing detecting anomalies in TLS encrypted traffic. Flow based analysis is therefore used as the industry standard for large scale network anomaly detection and the proposed project uses flow-based analysis techniques.

Summary of relevant published work (Literature review)

Problem landscape: why IoT anomaly detection is difficult.

2.1 Threat Landscape and Motivation

The massive expansion of the number of IoT devices from smart cameras and thermostats to voice assistants and industrial sensors poses a much larger threat landscape for attackers [1][5]. IoT devices tend to have weak default authentication credentials, poor firmware update policies, and limited built-in security [1]. The Mirai botnet attack analysed by Antonakakis et al. demonstrated the potential of using hundreds of thousands of easily-comprisable IoT devices for launching huge distributed denial-of-service (DDoS) attacks.[1] Several subsequent studies documented the rise of new malware types targeting various IoT devices, ranging from home automation hubs to smart locks and reinforcing the need for effective network-based intrusion detection solutions [5][6].

One of the difficulties in building such solutions is the lack of labelled data [5][6]. Real-world administrators never know beforehand what an attack looks like, attacks may evolve faster than the labelling process, and many events that might warrant a response in real-life situations would go unnoticed [5]. Additionally, IoT networks are highly heterogeneous in terms of their traffic characteristics, varying not only between device types but also based on firmware versions and patterns of usage [6]. As a result, the implementation of signature-based IDSs or any supervised learning techniques requiring labelled datasets becomes impractical [3][10].

2.2 Benchmark Datasets for IoT Network IDS

Various datasets are available for developing network IDSs for IoT devices, each with its advantages and drawbacks [4][10]. N-BaIoT by Meidan et al. includes traces of attacks conducted using nine commercial IoT devices against the Mirai and BASHLITE bots; its advantage lies in the detailed representation of device-specific attacks [2]. Yet, its limitations are evident: it is too focused on a single family of malware operating on a single type of device [10]. TON-IoT expands the scope to cover telemetry from sensors, logs and network traffic in smart cities and industry environments; yet, it adds to the complexity of data processing [10].

The most used dataset in recent studies is IoT-23, developed at Stratosphere Laboratory; it contains traces from 23 attack scenarios carried out using real consumer IoT devices (Amazon Echo, Philips Hue etc.) under both normal and various malware-controlled conditions; the latter includes flow logs produced using the Zeek IDS and representing a large variety of numerical characteristics useful for machine learning [4]. Given its size and public availability, IoT-23 became widely used in current IDS research; nevertheless, due to its lab-created nature and significant class imbalance present in some scenarios (benign and malicious flow numbers are unevenly matched), a network IDS trained on IoT-23 should take those issues into account [4][7][10].

2.3 Deep Autoencoders for Anomaly Detection

An autoencoder represents a neural network trained to transform its inputs into a compressed code representation (encoding) and then back into its original form (decoding); this is done in a way that minimises the difference between inputs and outputs (reconstruction error) [2] [6]. Training on a set of normal samples leads the model to learn a manifold of typical cases.[2] During inference, inputs that lie within this manifold result in low reconstruction error; inputs deviating from the normal are anomalous, hence have high reconstruction error [2] [6]. The use of a threshold applied to reconstruction error (typically, mean squared error) produces binary classifications for anomaly detection [6].

Deep autoencoders for IoT network anomaly detection are most famously demonstrated by Meidan et al., who managed to achieve near-perfect true-positive rate for Mirai and BASHLITE malware running on real devices by applying deep autoencoders trained solely on benign traffic [2]. A few other surveys dedicated to application of AI in anomaly detection on IoT and sensor networks also highlight autoencoders as one of the top choices, especially in cases where labelled data is insufficient or expensive to acquire [6][10]. A more extensive study on various deep-learning methods for IoT IDS shows that a professionally designed and fine-tuned autoencoder-based model can outperform popular machine learning approaches, such as Random Forests and SVM, on multiple datasets [10].

The appeal of autoencoders for IoT network intrusion detection lies in a few properties [6]. First, they enable unsupervised learning, i.e., training with no labels; this is particularly useful for applications where attacks are rare and labelled data is hard to come by [5][6]. Secondly, they can learn complex, non-linear dependencies between numerical characteristics; thirdly, inference is inexpensive and, therefore, allows real-time detection on cheap hardware [2]. Recent studies, however, have shown that under certain circumstances, autoencoders are unable to reliably discriminate between benign and malicious inputs; this means that not all anomalies result in high reconstruction error [19]. Therefore, careful threshold choice and testing are necessary.

2.4 Recent Deep Autoencoder Architectures for IoT and Network IDS (2020 – 2026)

Since 2020, quite a few publications focused on designing increasingly complicated deep learning architectures based on autoencoders to improve detection capabilities.[8][10]. One of the examples is the AE-LSTM model, which combines a stacked autoencoder with an LSTM classifier and achieves high F1-scores on NSL-KDD; yet this approach depends on having labelled attack data [10]. Another is an autoencoder-lstm-ensemble method, MSCNN-LSTM-AE, based on convolutional autoencoder and LSTM-encoder; this approach performs well on multiple benchmarks, such as NSL-KDD, UNSW-NB15 and CICDDoS2019; however, its complex design makes it harder to deploy to IoT devices [17].

Another set of papers considers probabilistic approaches to deep learning-based IDS. An ensemble model with two kinds of probabilistic autoencoders, CNN-VAE and its modifications, yields modest performance improvement over individual models; yet, their output is more difficult to analyse, as the latent layer relies on probability distributions instead of deterministic values [16]. A similar approach is employed by CUMAD, which integrates autoencoder-based IDS with sequential hypothesis testing to reduce the number of false positives in botnet scenarios; yet, as a side effect, this increases alert latency [11]. A different direction of research is concerned with finding ways to enhance the reliability and performance of IDS.

Some of the proposed enhancements include special autoencoder architecture and feature selection. A logarithmic deep autoencoder in combination with XGBoost learns to normalise the data using a specially designed layer to obtain impressive accuracy on UNSW-NB15 and CICIDS2017; yet the downstream classifier is supervised, and the data consists mostly of non-IoT instances [14]. An ANOVA-based feature selection approach together with deep autoencoders enables Rhachi et al. to achieve modest results on IoT-23 and NSL-KDD [13]. Sparse autoencoders in combination with feed-forward or deep networks perform well in edge computing; yet this approach is limited by a necessity to train a classifier separately [8][9].

In addition, researchers attempted to tackle problems associated with the constant change of underlying data. One approach was the use of a combined autoencoder-ensemble-drift-detection mechanism called strAEm++DD; it performs well but requires a lot of resources and provides no visibility of drift detection [12]. All research on IoT network IDS assumes the absence of concept drift and trains static models on datasets [5][10]. A separate approach focuses on training autoencoders for lightweight IoT devices (TinyML); these models require aggressive pruning and quantisation to reduce their size.[9][15] Another recent direction in this field involves the use of autoencoders in zero-shot and low-shot intrusion detection [16][18].

2.5 Evaluation Metrics for IoT Network IDS

Imbalance is a fundamental problem in network intrusion datasets: there are typically more benign flows than malicious or vice versa, especially when dealing with curated datasets, as in IoT-23 [4][10]. Therefore, metrics like accuracy may be misleading in this setting [7]. The authors of a recent publication state that the Precision-Recall curve provides a better representation of model performance in cases with an under sampled positive class, which implies a clear trade-off between false positives and false negatives [7]. Accordingly, the metrics recommended by recent publications are the area under PR curve (PR-AUC) and F1-score; ROC AUC is also sometimes used [6][7][10].

However, the autoencoders-based network IDS literature fails to address the issue: most articles do not report PR-AUC or simply focus on accuracy, and others do not provide a comprehensive set of metrics [6][10]. For instance, the models described in works on attention-based IDS achieve high recall; yet, their poor precision means generating a vast number of false positives, which is hardly sustainable for humans to manage [16][18]. There are also attempts to lower false positive rate, for example, with the help of sequential testing in CUMAD; yet this introduces extra latency, rather than helping to tune threshold values interactively [11].

2.6 Critical Gaps and Areas for Improvement

Although promising, research on IoT intrusion detection has several gaps that are important to fill. Firstly, recent works challenge the reliability of autoencoder-based IDS, demonstrating a case when anomalies were reconstructed equally well as normal flows [19]. This means that a significant part of literature ignores the problem or proposes inadequate mitigation measures.

Section 3. Tools and Methods

Rational for technical tools selected

Python as the Principal Implementation Language

There is straightforward evidence for choosing Python as the principal programming language used for both deep learning architecture definition and model training, engineering of the data pipeline, and development of the data visualisation web application dashboard itself. The dominance of the Python ecosystem in machine learning practise in 2025 cannot be overstated: more than 80% of AI and machine learning job postings require or prefer familiarity with Python and it has the richest selection of libraries dedicated to data processing and deep learning compared to other contemporary environments. Crucially, from the perspective of this Final Year Project work, Python's interpreted nature, and interactive programming style (eg., in the context of Jupyter notebooks) allow for rapid prototyping cycles as well as reproducible analysis where the data preprocessing, architecture building and evaluation steps can be executed iteratively in the same session.

TensorFlow / Keras vs PyTorch: Justifiable Choice

In this project, the use of the TensorFlow stack with Keras is the appropriate choice because of the following reasons. First, with the introduction of the native TensorFlow version 2.x Keras API, the architectural boilerplate for building deep neural networks has decreased considerably: it is possible to build a fully connected deep autoencoder and train it in less than thirty lines of code using `tf.keras.Sequential`, `model.compile()`, and `model.fit()`. In addition, TensorFlow offers built-in reproducibility capabilities, allowing for setting seed and enabling op deterministic execution through `keras.utils.set_random_seed()` and `tf.config.experimental.enable_op_determinism()`. These tools are critical for providing deterministic behaviour in the model and allowing for reliable evaluation of the experimental procedure in an academically sound fashion. Finally, the TensorFlow deployment options including TensorFlow lite (for edge device runtime) and TensorFlow serving (for model persistence), provide more complete options compared to PyTorch. In addition, the interface for saving and loading TensorFlow Keras models via `model.save()` and `load_model()` respectively facilitates seamless transfer of the trained model artefacts into the Streamlit application environment.

The following table summarises the above:

Criterion	TensorFlow/Keras	PyTorch
Api simplicity	High – Keras Sequential API	Moderate – explicit loop required
Reproducibility tooling	Native – set_random_seed, op determinism	Manual – requires seed setting across multiple libraries
Edge deployment	TFLite – mature, production standard	ONNX export – less integrated
Research community adoption	Declining in academia; strong in industry	Dominant in research (>55% production share Q3 2025)
Dashboard integration	Direct – Keras SavedModel loadable in Python	Direct – torch. load equally viable
Verdict for this project	Preferred – simpler API, native reproducibility, mature ecosystem	Suitable for research; excess flexibility unnecessary here.

Streamlit vs Flask: Justification for Rapid Prototype Dashboard Delivery

Criterion	Streamlit	Flask
Development speed	Extremely fast – pure python	Moderate to slow – requires HTML/CSS/JS
ML model integration	Direct – python – native no serialisation overhead	Requires API endpoint construction and serialisation
Interactive widgets	Built in – sliders, charts, tables	Manual – JavaScript or third-party library required
Deployment complexity	Low – streamlit run app.py	High – WSGI server, Nginx/Apache, often Docker
Production scalability	Limited not suitable for multi user enterprise	High – full web application capability
Verdict for this project	Preferred – prototype velocity and ML – native integration	Appropriate for production; excess complexity for a prototype

Rationale for research methods used.

Research paradigm

The current research adopts a quantitative, experimental research paradigm, the most suitable methodology for investigating software engineering problems related to the objective measurement of the performance of a computational system. While a qualitative research paradigm will produce an interpretative finding derived from interviews or observation, an experimental paradigm produces objective results measured as performance metrics and presented numerically, namely as F1-Score, PR-AUC, and false positive rate, which can be verified, replicated, and compared with a baseline. Moreover, unlike the findings from a qualitative paradigm, the experimental research is directly aligned with the posed research question, asking to what extent the system succeeds in performing effective anomaly detection [21].

Dataset selection: IoT-23 compared with other benchmarks.

The IoT-23 dataset has been chosen to be the main test benchmark among three other popular IDS datasets: NSL-KDD, UNSW-NB15 and IoT 23 [22].

Firstly, NSL-KDD has received harsh criticism from the community for its redundant records, artificially engineered class distribution, and traffic behaviour reflecting outdated, unrealistic network operations, rendering a model trained on it an invalid generaliser for real IoT traffic. UNSW-NB15 represents a notable step towards realism, including modern attack types and traffic behaviour; the authors have established it as a superior substitute for NSL-KDD in evaluating NIDS. However, it still represents network traffic from a general enterprise lab environment rather than from actual IoT devices [23].

IoT-23 has been chosen because it is the only major publicly accessible benchmark of real network traffic recorded from actual IoT hardware, including the Philips HUE smart lamp, Amazon Echo and Somfy Smart Door Lock. IoT-23 includes 23 different scenarios compromising both normal and attack traffic. This makes it an optimal choice of the dataset for the current research since the autoencoder will learn a normal behaviour model, which reflects the heterogeneous structure of real IoT traffic [39].

Evaluation Metrics: F1-Score and PR-AUC vs Accuracy

In selecting F1 score and Precision-Recall AUC as the main evaluation metrics and ignoring the raw classification accuracy score, one needs to consider that the dataset has an imbalanced distribution of classes, namely most flows are normal while attack flows are relatively few. Under these conditions, a naïve model that predicts normal class for all flows will achieve near perfect accuracy score whilst detecting no anomalies whatsoever, rendering the result statistically valid but functionality meaningless [21].

However, F1 score mitigates this problem by considering both false negative and false positive rates. This metric calculates a harmonic mean of Precision and Recall rates. Meanwhile, precision recall area under curve considers how good the detector is at detecting attack flows at all thresholds. These metrics have been recommended by a 2025 arXiv review of absolute evaluation measures to be “often the most dependable for general-purpose evaluation in imbalanced datasets”. Meanwhile, Milvus (2026) explicitly mentions the advantage of PR-AUC over ROC-AUC: PR-AUC detects “weaknesses in detecting subtle attacks due to low precision”. Lastly, as previously mentioned, the false positive rate is considered to address the operational safety concern posed in the Legal and Ethical Issues section [24].

Baseline Comparator: Isolation Forest

Isolation Forest is used as a comparator in the current research for two methodological reasons. Firstly, this unsupervised algorithm is based on the same paradigm of working, as an autoencoder. Hence it is a like for like comparator, making sure that the relative contribution of the deep autoencoder architecture in detection performance is isolated. Secondly, Isolation Forest is an established reproducible benchmark, which is used for testing neural networks in multiple peer reviewed studies [16].

Reproducibility Strategy

Reproducibility is ensured via four methods. Firstly, a fixed random seed is set for both the Keras backend and the TensorFlow engine through `keras.utils.set_random_seed()` and `tf.config.experimental.enable_op_determinism()`. This ensures deterministic results in each training attempt. Secondly, the IoT-23 benchmark is retrieved from the Zenodo repository [5]; thus, ensuring immutability and traceability of the benchmark used in the experiment. Thirdly, model weights are saved using `model.save()`, resulting in a portable SavedModel artefact, which could be loaded and evaluated independently of the environment it has been trained in. Finally, all the required packages are specified in a resolute `requirements.txt` file.

Rationale for Project Management Tools and Product Lifecycle used.

Alignment between Agile principles and the ML Development Cycle

This approach to software engineering projects is best suited for implementing deep learning software due to the nature of its epistemological fit. Unlike conventional software development projects where exact requirements can be defined, the process of creating a neural network is much less deterministic and iterative in nature since it is impossible to know what will be the best model architecture, what will be the best values of the hyperparameters, how data should be pre-processed and at what evaluation threshold a model can produce the best predictions [25].

This epistemological match between Deep Learning projects and the Agile software methodology was discussed in detail in the scientific literature. The systematic mapping paper published on arXiv in 2025 discusses the main adaptations that Agile software engineering techniques have taken to address the unique characteristics of the machine learning process. Among such adaptations mentioned are, for example Iteration Flexibility allowing for data processing and model training through flexible sprints and MVM (Minimum Viable Model) representing testable hypothesis and model performance [25].

Sprint structure for Iterative Hyperparameter Tuning

The most technical benefit of using Scrum methodology on this project comes from the ability to easily tune hyperparameters within each Sprint cycle. The technique of hyperparameter tuning is used when important model parameters such as bottleneck size k , learning rate a , batch size and numbers of encoder/decoder layers are optimized through multiple rounds of experiments [26].

According to the Scrum methodology, each Sprint (lasting one-two weeks typically), consists of the following sequence of stages:

Sprint Backlog Item > Train > Evaluate > Retrospect > Refine.

It corresponds to the hyperparameter search procedure defined by AWS Machine Learning Lens (2024): Define Configuration > Training Pipeline (Data Pipeline) > Performance Evaluation (F1-Score and PR-AUC metrics). A course to fine approach to tuning the hyperparameters, which starts broad parameter ranges and gradually moves to the optimum, can only be conducted iteratively [26].

For the present project, the sprint to hyperparameter mapping is operationalised as follows:

Sprint	Primary Model Focus	Key Hyperparameter under Tuning
Sprint 1	Data pipeline, baseline autoencoder	Input normalisation strategy, train/test split ratio
Sprint 2	Encoder depth and bottleneck	Number of hidden layers, bottleneck size $k \in \{8,16,32\}$
Sprint 3	Reconstruction threshold calibration	MSE threshold T at 95 th vs 99 th percentile of validation
Sprint 4	Optimiser and regularisation tuning	Learning rate, α , dropout rate, L2 regularisation coefficient
Sprint 5	Streamlit dashboard integration	Real time inference pipeline, threshold slider range
Sprint 6	Baseline comparison and evaluation	Isolation Forest contamination parameter; PR-AUC benchmarking

Waterfall: Why it falls for Deep Learning Projects

The essential incompatibility between the Waterfall methodology and the deep learning project is built into the nature of the latter itself. In the Waterfall approach, the project progresses through a series of sequential phases: Requirements, Design, Implementation, Verification and Maintenance, with each one completed and signed off before starting on the next. Such a structure assumes that the behaviour of the system can be known in advance and specified in design, which cannot be done for a deep autoencoder that learns to distinguish between the target class and noise through training its parameters and thresholds [27].

This will have disastrous consequences for our project. According to the Waterfall structure, we would have to define in the Design phase the desired values of bottleneck dimensionality, learning rate, and reconstruction threshold – without ever having trained the model yet – and once discovering that the first attempt produces inadequate PR-AUC, submit a change request and formally repeat the Design phase. Indeed, numerous studies on the success rate comparison between Waterfall and Agile methodologies show that projects following the former methodology fail completely in 59% of cases, whereas Agile – based initiatives fail only in 11%, due to the inflexibility of Waterfall’s structure. Moreover, according to Waterfall’s sequential nature, all stakeholder validation will happen during the final phase, so any issues found during this process by the administrator regarding Streamlit dashboard will come too late [28].

Risk Mitigation Through Incremental Delivery

Another methodology benefit of scrum is its continuous, sprint-based risk management process. For example, considering our deep autoencoder project, three specific technical risks deserve mention:

[29]

- **Model collapse risk** – the autoencoder converges on a near identity function and thus generates uniform low reconstruction error, rendering the anomaly detection threshold meaningless. This failure mode will be identifiable in Sprint 2 under scrum and correctable via architectural modification prior to subsequent sprints being committed [30].
- **Data leakage risk** – anomalous data points accidentally included in the training dataset skew the normal data model. Sprint 1's special data pipeline sprint separates this risk before model training occurs.
- **Threshold sensitivity risk** – a fixed threshold t yields an unacceptably high false positive rate during live deployment. Sprint 3's special threshold tuning sprint, in combination with the adjustable slider in the Streamlit web interface, forms a sprint-specific risk mitigation approach.

In each instance, the sprint barrier serves as a structured risk identification point at which the risk can be identified, assessed and mitigated prior to spilling over into downstream activities, forming a risk management characteristic that the Waterfall process's linear gate design cannot emulate. Thus, agile is not just a faster development process; it is a superior architecture of risk management for an effort whose primary technical innovations are discovered experimentally.

Legal, ethical and professional issues

GDPR Compliance and Network Traffic Data

Firstly, implementation of the anomaly detection system using live network traffic implies significant compliance obligations in accordance with the UK GDPR and EU GDPR (Regulation 2016/679). The metadata about network traffic consisting of source and destination IP address, ports, timestamps of connection and byte sizes represent personal data under GDPR if this information can be used alone or in combination with other identifiers for the purpose of identifying a natural person. The ICO explicitly states that the IP addresses constitute personal data in the context of IoT monitoring; therefore, any system that uses this type of data falls under all the GDPR regulations in relation to rights of individuals and controllers [31].

The Ethics of False Positives in Critical IoT Environments

The moral element of the problem is especially critical in cases when IoT devices are intended to perform safety critical tasks. False positive – an alert generated from normal traffic – has a fundamentally different level of significance depending on the application context. If a false positive alert is triggered in a home lightning system, there will be slight damage done. If a false positive happens in a healthcare IoT deployment, where potentially flagged devices might be patient monitors, infusion pumps and surgical controls, then the resulting interruption in treatment might become fatal for the patient [32].

Such a distinction makes it a professional engineering ethics case study according to the ACM Code of Ethics and the BCS Code of Conduct. Both codes stipulate that software engineers' highest responsibility is for the protection of the public well-being, health and safety. The evaluation of IDS deployment in a safety critical system as performed by the University of Glasgow highlights the concern of "an IDS may trigger itself a denial of service, if it is too sensitive and therefore triggers alarms even in the presence of nominal behaviour". Such problems are solved in the current research via two approaches. On one hand, the PR-AUC metric is chosen as the main evaluation criterion since it explicitly shows the trade-off between the rate of false negatives and false positives and precludes picking an inappropriate threshold to maximise the latter. On the other hand, the possibility of

adjusting reconstruction error threshold in the Streamlit dashboard implements the human in the loop approach which is a necessary part of the principle of professional responsibility applied to automated detection in safety critical systems [2].

IoT-23 Dataset: Privacy Risk Mitigation

The choice of the IoT-23 dataset for training the models and assessing their performance should be regarded as a deliberate step taken to mitigate the privacy concerns raised above. The IoT-23 consists of 23 samples of network traffic captured by the hardware components of IoT devices and encompasses more than 325 million labelled flows, published freely by the Stratosphere Laboratory, CTU University. Importantly, IoT-23 does not involve the processing of users' personal data in the GDPR sense and is therefore not regulated by it; no personal information is available in its raw version [40].

Privacy protection involves several layers. From a data standpoint, IoT-23 is made available in Zeek/Bro connection logs – a flow-based representation of data from which personal information cannot be reconstructed. From an organizational perspective, IoT-23 has been published through a funded academic research programme bound to comply with ethical standards, and its use in machine learning research has been supported throughout subsequent literature without questioning its privacy-friendly nature. Recent studies focused on differential privacy confirm that IoT-23 fulfils moderate privacy budgets in line with k-anonymity and l-diversity measures [41].

Risk Register

#	Risk	Category	Likelihood	Impact	Severity	Mitigation strategy	Contingency
R1	Model Overfitting — the autoencoder memorises training flows rather than learning generalised normal-behaviour representations, producing low reconstruction error on anomalous test flows and degrading detection performance.	Technical / ML	Medium	High	High	Apply Dropout layers (rate 0.2–0.3) between encoder and decoder layers to prevent co-adaptation of neurons; apply L2 weight regularisation; monitor validation loss curve per epoch for divergence from training loss; use early stopping with patience=10 to halt training at the optimal generalisation point [30].	Reduce model depth; increase bottleneck dimensionality k ; retrain on augmented dataset in Sprint 4.

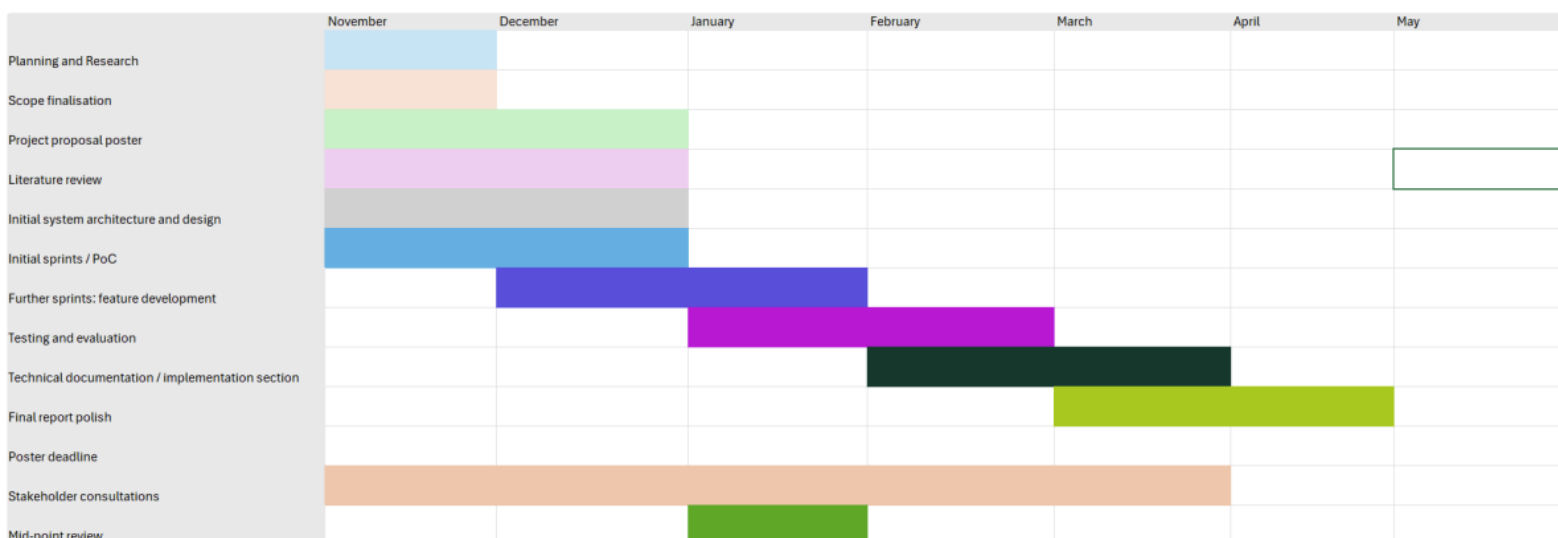
R 2	Data Imbalance — IoT-23 contains a heavily skewed class distribution where normal flows vastly outnumber malicious flows, causing the autoencoder to optimise reconstruction of normal data whilst failing to produce sufficiently elevated reconstruction error on anomalous minority-class records.	Technical / Data	High	High	Critical	Apply threshold calibration at the 95th–99th percentile of validation reconstruction error rather than a fixed scalar value; use SMOTE (Synthetic Minority Oversampling Technique) or under sampling of normal flows during pipeline preprocessing to rebalance training batches; report PR-AUC rather than accuracy to surface class-imbalance effects in evaluation [4].	Switch to weighted MSE loss function that penalises reconstruction error on anomalous samples more heavily; apply log-scaling to reconstruction error distribution.
--------	---	------------------	------	------	----------	--	--

R 3	Hardware Constraints for Dashboard — the Streamlit dashboard performing real-time inference on IoT-23 flow records may experience latency or memory pressure when running on a standard development laptop, particularly during large-batch reconstruction error computation.	Technical / Infrastructure	Medium	Medium	Medium	Implement batch inference in the Streamlit pipeline rather than record-by-record processing; load the saved TensorFlow model once at application startup using <code>st.cache_resource()</code> to prevent redundant reloading on each widget interaction; limit the default visualisation window to the most recent 1,000 flow records with a user-adjustable slider [35].	Reduce model complexity (fewer layers, smaller layer widths) to decrease inference memory footprint; offload heavy computation to a background thread using Python's concurrent Futures.
--------	--	----------------------------	--------	--------	--------	--	--

R 4	Project Timeline Delays — unexpected technical challenges in data pipeline construction, hyperparameter tuning, or dashboard integration may compress later sprint allocations, risking incomplete evaluation or submission deadline breach.	Project Management	Medium	High	High	Maintain a Trello sprint backlog with clearly defined sprint exit criteria and weekly review checkpoints; build a 2-sprint contingency buffer into the Gantt chart by completing core model training (Sprints 1–4) before the dashboard integration sprint (Sprint 5); prioritise the Minimum Viable Model (MVM) — a	De-scope the Streamlit threshold slider to a static pre-set value if Sprint 5 is at risk; submit the console-based evaluation pipeline as the primary artefact with the dashboard as an extension.

						functional autoencoder with logged metrics — as the Sprint 2 deliverable to ensure a submittable artefact exists early [25].	
--	--	--	--	--	--	--	--

Gantt Chart



The Gantt chart depicts the expected schedule of completion of the various stages of the project, from research to design, development, testing documenting and final changes. The chart indicates that the project had been well organized on a schedule and the activities conducted systematically during the school year. It is useful in addressing the project management part of the report.

Section 4. Requirements/Specifications

System Overview and Architecture

The constructed system architecture is a pipeline process composed of four main stages: data intake and preprocessing, unsupervised learning and anomaly detection using a deep autoencoder, classical Isolation Forest baseline model, and Streamlit interactive user interface for network managers. IoT-23 Zeek connection logs are ingested and converted to a clean, normalized set of features. Then the autoencoder model is trained solely on benign flows, creating an efficient – low dimensional representation of normal IoT activity, while the classical Isolation Forest algorithm learns a model using the same feature space. During deployment, both models core incoming traffic flows, and results are presented in the interactive Streamlit dashboard for investigation and decision making by the network manager.

Requirements and Stakeholders

The primary user of the system is the network / security administrator whose responsibility is to supervise the IoT devices connected within a local network, which could be a university, an organization or a smart home setup. The primary user needs timely notifications about suspicious device activity without necessarily having to have expert knowledge in machine learning. Other users include security analysts who can conduct more detailed investigations of suspicious flow activity and organizational leaders who consider IoT risks holistically.

Functional Requirements

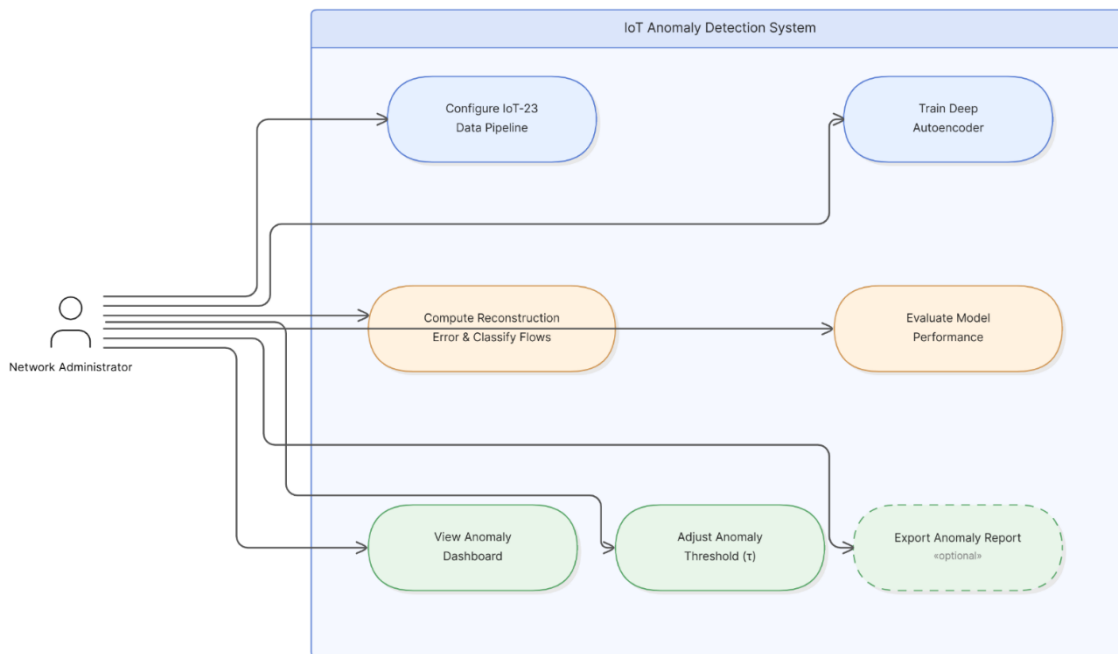
Upon ingestion of the network records, the system will calculate the reconstruction error of that flow by using the pre-trained Deep Autoencoder model.

If the reconstruction error of the flow is higher than the specified threshold value of T , then the system will mark that flow as anomalous and generate an anomaly output.

In case the Isolation Forest model is enabled, the system will assign a binary anomaly output to each flow independent of the autoencoder output.

On the loading of the dashboard, the system will display the histogram showing the distribution of the reconstruction errors for all flows from the loaded dataset.

Figure 4.2 Use case diagram for the IoT Anomaly Detection System (Network Administrator actor)



It shows the crucial interactions that occur between the network administrator and the system. The main use cases include the uploading of IoT-23 flow data, the distribution analysis of the reconstruction error, the interactive adjustment of the anomaly detection threshold, and the review of each suspicious flow. The comparative use case provides the network administrator the ability to compare the results of autoencoder and Isolation Forest algorithms.

Non-Functional Requirements

Performance

The system will process and score each network flow after receiving it within 500 milliseconds for anomaly detection without GPU capacity.

The histogram and anomaly table on the dashboard will be updated within 2 seconds after setting a new threshold value on the standard hardware.

Usability

The user should be able to set the threshold for anomaly detection and see results without any retraining of the model or changing the codebase.

All elements of the dashboard should have understandable English Language labels to clarify what they do and what information they display to a user with no ML background.

Success Criteria

Criterion	Target
SC1 – Autoencoder F1-score on IoT-23 test set	>/ 0.80
SC2 – Autoencoder PR-AUC on IoT-23 test set	>/ 0.85
SC3 – Autoencoder recall (true positive rate)	>/ 0.85
SC4 – Autoencoder false positive rate	</ 0.15
SC5 – Autoencoder outperforms Isolation Forest on F1-score	Autoencoder F1 > IF F1
SC6 – Dashboard threshold slider updates in real time	</ 2 seconds response
SC7 – System runs inference without GPU	Confirmed on CPU hardware
SC8- All UAT tasks completed by test users without error	100% task completion rate

Section 5. Solution design

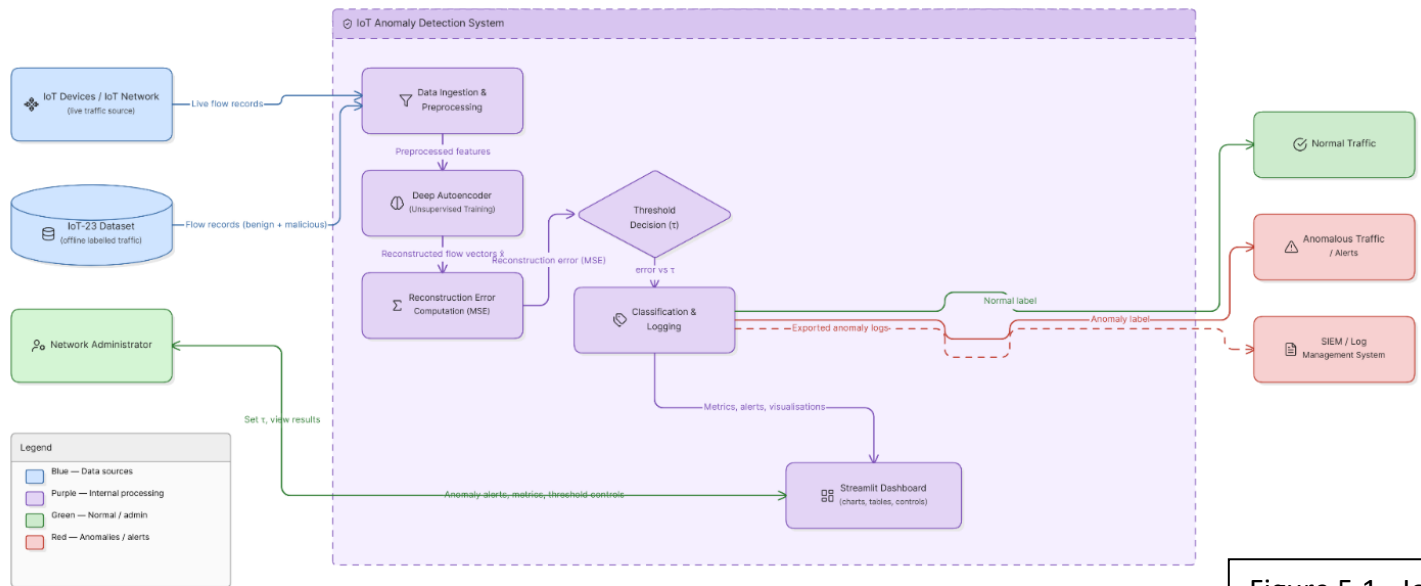


Figure 5.1 - IoT Anomaly Detection System Data Flow Diagram

The figure illustrates the IoT anomaly detection pipeline from end to end. The IoT or IoT-23 flows are processed by preprocessing and then running a deep autoencoder on them, with the output being compared with a configurable threshold to determine whether the flow is normal or anomalous. This information is presented to the network administrator via a Streamlit dashboard, and anomalies may be relayed to a SIEM or other external alerting mechanism.

5.1 Design Overview

The section outlines the design of the proposed IoT network anomaly detection system, based on the functional and non-functional requirements outlined in Section 4. Each important architectural choice and algorithm is chosen due to being supported by findings of the related literature review from Section 2, which provides scientific basis for the decisions made instead of arbitrary technical choices.

The guiding principle of the system design is the use of unsupervised learning of anomalous behaviour through reconstruction error. In the proposed system, the deep autoencoder is the only ever trained on flows determined to be known benign, which gives it a chance to learn a model of normal IoT flow behaviour. Then at the test stage, when the flow is anomalous because of a compromised or faulty IoT device, its reconstruction would produce a high mean squared error (MSE) – the anomaly score used by the system. This design choice was made instead of using unsupervised classifiers, since, IoT systems lack labelled datasets of attacks for training models, as several independent surveys have concluded [2].

Isolation Forest Baseline model is incorporated alongside the autoencoder model to serve as a direct comparison point. The two models have been evaluated using the IoT-23 dataset that has been split using the same preprocessing pipeline to ensure that all the differences in performance are due to the structure of the models themselves and not because of variations in how the data was pre-processed. Outputs for both the models are provided through a Streamlit web interface [8].

5.2 System Architecture

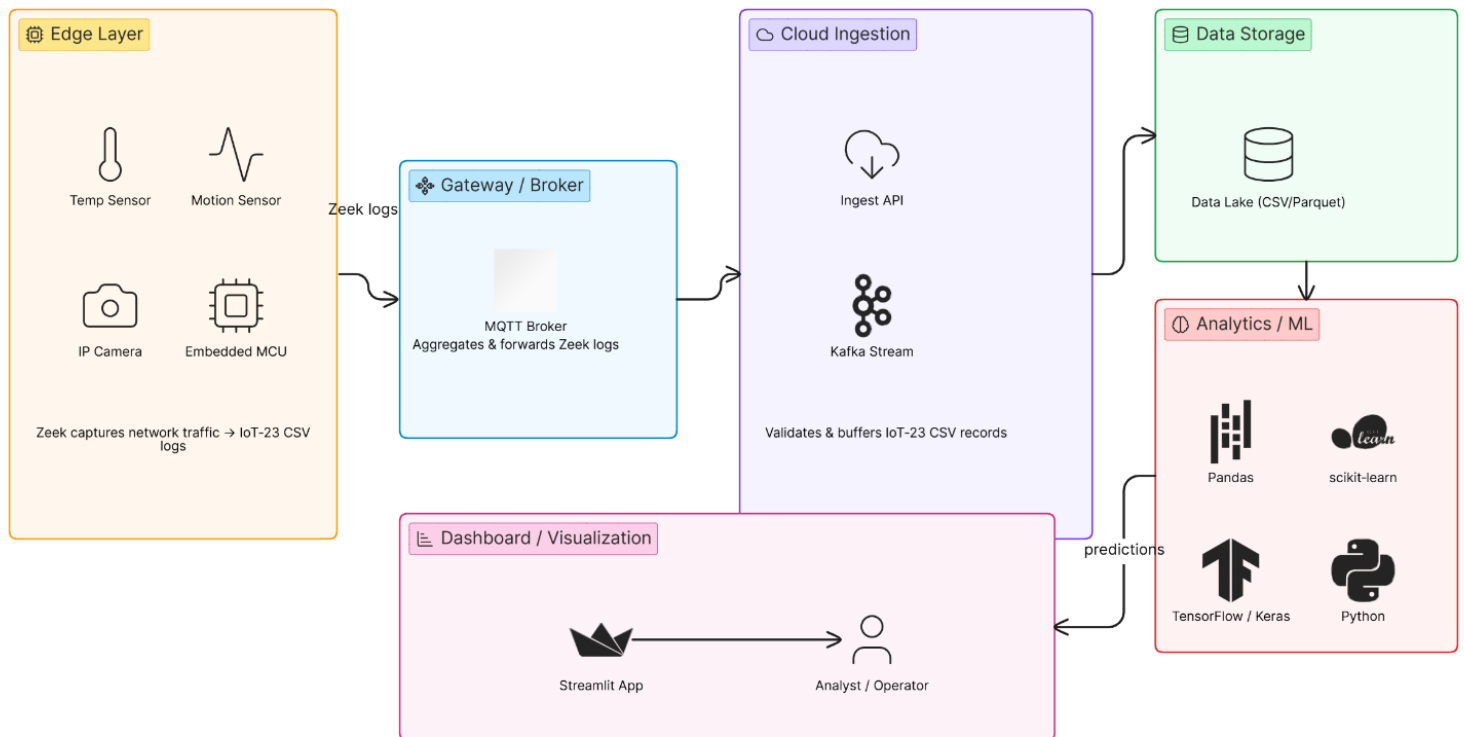


Figure 5.2 High-level system architecture for the IoT network anomaly detection system, showing five processing layers from data ingestion to dashboard output

Layer 1

Layer 1 – Data digestion: the network flow records collected from IoT-23 dataset are used as the main source of data in our work. A flow record represents a connection level statistic generated by the Zeek network analyser and contains the following features – connection duration, byte counts (originator and responder), packet counts, protocol type, and connection state flag bits. In production mode, this layer can be easily substituted by a network tap or SIEM output feed without any alterations in the subsequent processing chain [39].

Layer 2

Layer 2 – preprocessing: the raw flow records are pre-processed prior to running models. The columns that do not add any predictive power are dropped (timestamps, IP addresses, ports). To avoid spurious correlations with devices and not with behavioural patterns, non-informative columns are stripped away. The missing values are imputed by median. All features are standardized using the StandardScaler fit only on training data to avoid leaking any information from the validation and test splits into the preprocessing stage [4].

Layer 3

Model inference: a pre-processed feature vector serves as the input for both the trained deep autoencoder and Isolation Forest. The autoencoder generates the reconstruction of the feature vector and calculates the mean squared error of this reconstruction relative to the original vector. Isolation Forest calculates the anomaly score independently using an ensemble of isolation trees.

Layer 4

Anomaly classification: based on the calculated MSE value, the autoencoder makes a binary decision: flows with $MSE > T$ get the label ‘anomalous’, while the rest are considered ‘benign’. The Isolation Forest generates another binary classification for comparison purposes. More details about the approach to calibrating the threshold T are provided in section 5.4.

Layer 5

Streamlit dashboard: the results produced by all layers are visualized by the Streamlit web application. This includes feature vectors MSE values, binary classification and various aggregated statistics. With this application, the network administrator can analyse the detected anomalies, upload new CSV flow data, tune the classification threshold k in real time, and get immediate feedback on precision, recall, F1-score and PR-AUC values.

5.3 Data pipeline design

Data Pipeline: Anomaly Detection on IoT-23

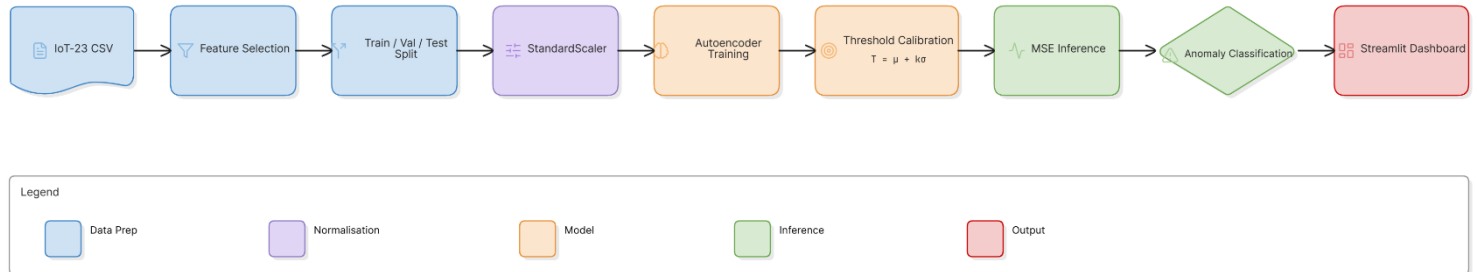


Figure 5.3 above shows the end-to-end flow of data through the system pipeline, from raw IoT-23 CSV files through to the final anomaly label output displayed on the dashboard

This procedure occurs in eight subsequent stages as follows:

Loading of raw data

CSV files containing Zeek connection logs from IoT-23 are loaded as a Pandas DataFrame. The label of the ground truth (benign / malicious) is extracted from this DataFrame and kept separately to perform evaluation later. The label is never passed to the neural network model [5], [5].

Feature selection and cleansing

All non-numerical columns (source IP, destination IP, port numbers) along with timestamps are excluded. Remaining NaNs are imputed using column-wise median of numerical features to maintain the sample size while avoiding any possible leakage due to the distribution of input features [4].

Train / validation / test split

The initial DataFrame is split into train (70% of samples with only benign flows), validation (15% of samples with only benign flows), and test (15%) sets. Only the benign flows should be present in train and validation sets since training an autoencoder for unsupervised anomaly detection via reconstruction error relies on not observing any anomalies during training [2].

Normalisation

The StandardScaler is fitted on the train part of the data only and applied to validation and test data sets. Fitting the scale only on training set avoids information leakage caused by passing parameters of the validation and test distribution into the model. This step is critical for unbiased results [38].

Autoencoder training

Training is done on the normalised train split with early stopping based on the validation reconstruction loss. Fully connected encoder layers compress the input representation down to a bottleneck, which is then expanded by fully connected decoder layers into the original feature vector. The training objective is to minimise the reconstruction error with MSE loss function [2].

Threshold calibration

After the training is completed, the autoencoder is applied to the validation set consisting only of normal flows to obtain the distribution of the reconstruction losses on the normal data. The anomaly threshold T is calculated according to section 5.4.

Inference and MSE scoring.

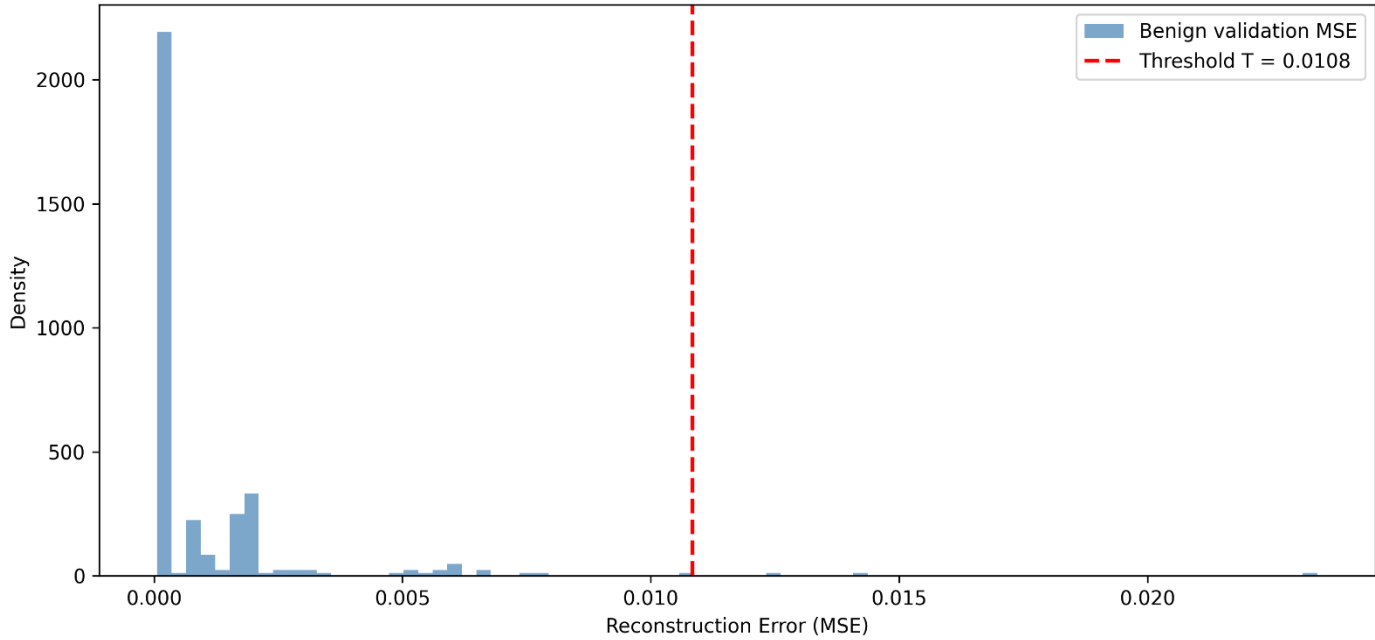
All flows from the test set are passed through the autoencoder to receive the anomaly scores defined as MSE on the difference between the original and reconstructed representations. At the same time, Isolation Forest is applied to the same flows [8].

Evaluation and dashboard generation

Labels generated by the model based on whether MSE was greater than the threshold T and predictions provided by IoT-23 are compared to calculate such metrics as precision, recall, F1-score and PR-AUC. These metrics suggested by Saito and Rehmsmeier are chosen to evaluate the performance of an unsupervised classifier since PR-AUC works better on imbalanced problems.

5.4 Threshold Design

Figure 5.4 - Reconstruction Error Distribution (Benign Validation Set)



The anomaly threshold, T is one of the most operationally relevant choices that have to be made when designing a detection system based on reconstruction error. An incredibly low choice of T results in many false alarms being triggered, while a high threshold results in attacks being missed. Thus, designing the threshold T involves finding the optimal trade-off between detection recall and operational precision [7].

The threshold is calculated using the validation dataset, which consists only of benign flows. Once trained, the model is used on the validation dataset, and the reconstruction errors are recorded to compute a threshold that is defined as:

$$T = \mu + k \cdot \sigma$$

Where μ represents the average reconstruction error of benign flows in the validation set, σ denotes the standard deviation of these errors and k is a tunable sensitivity parameter. By default, $k = 3$ is taken, which under the Gaussian assumption puts T three standard deviations above the average error of benign flows and thus covers 99.7% of the normal flows in the benign band. This threshold calculation aligns well with the one proposed by Meidan et al. In their landmark paper N-Balot, where in the reconstruction error of benign flows is employed to compute the detection threshold [2].

The role of k is revealed as an adjustable parameter for the Streamlit dashboard. With the increase in k , the threshold will be increased, which means the decrease in false positive but the reduction in recall. On the other hand, with the reduction in k , the sensitivity will be

increased, but there will be some false alarms. These consequences are demonstrated through a live demonstration using various k values in the Streamlit dashboard [7].

This formula was chosen over the percentile of validation errors approach (for example, the 95th or 99th percentile) since the mean plus sigma method is statistically motivated, has an easily comprehensible meaning of standard deviations and scales reliably to datasets with varying ranges of absolute MSE values, without needing to recalculate the percentile constant specific to each dataset. It also establishes a clear connection between the threshold value and the statistical nature of the normal traffic, thus making the entire decision-making process traceable [18].

5.5 Design Decisions

The decisions related to the technological aspects of the solution were made based on the analysis and comparison of alternative solutions considering the conditions of the IoT anomaly detection problem. The design decisions together with the alternatives and justifications of the selection are shown in Table 5.1. All decisions are based on the literature review in section 2.

Component	Decision made	Alternatives considered	Justification
Primary detection model	Deep autoencoder	Supervised classifiers (Random Forest, SVM, CNN); LSTM Autoencoder; One class SVM	Autoencoders do not need any labelled data to train, which cannot be available in practice in IoT environments. Meidan et al., for instance, achieved 100% true positive rate on detecting botnets in attacks based on the use of autoencoders with benign traffic only. LSTMs have increased training complexity but do not provide any accuracy improvement for flow data [2].
Baseline model	Isolation Forest	Local outlier factor (LOF); One – Class SVM; statistical Z - core	Isolation forest has time complexity of $(O) n$ and can work efficiently in high dimensional space; moreover, it is considered as the most cited baseline for anomaly detection algorithms. Liu et al. showed that I Forest algorithm performs better than LOF algorithm and distance-based algorithms in terms of AUC and time complexity [8].

Dataset	lot-23	NSL-KDD; CICIDS2017; BoT-IoT	The IoT-23 dataset comprises actual traffic data from IoT devices in 20 malware attacks and 325 million labeled flows; hence, it is ecologically valid because it has been taken from natural settings rather than from synthetically generated or corporate data sets. NSL-KDD is more than 25 years old, and it does not have any traffic flow specific to IoT devices [39].
Threshold method	$T = \mu + k\sigma$ on benign validation set	Fixed percentile (95th/99 th); Otsu's method; Fixed constant	Mean plus sigma approach is statistically justified, understandable and parameterizable using the parameter k. Fixed percentile techniques are distribution – dependent, meaning that they are influenced by the characteristics of each data set; they cannot be analytically interpreted either. The selected technique relates the threshold value with the statistical characteristics of the benign traffic distribution.
Frontend / UI	Streamlit	Flask + HTML; Dash; No UI	With Streamlit, one can quickly build highly interactive Python based dashboards without any need for front end engineering whatsoever. It offers native support for live data refreshment and interactive parameters, which makes Streamlit very ideal for the FYP application, especially for a single developer system.
Normalisation method	StandardScaler (fitted on training set only)	MinMaxScaler; No normalisation; RobustScaler	The StandardScaler is the most used scaler when dealing with neural network inputs that have Gaussian like distribution properties. Training the scaler on the training data only avoids any leakage of information from the validation or test data distributions. The MinMaxScaler is highly influenced by the presence of outliers in the IoT traffic data.

5.6 System interaction (UML Sequence Diagram)

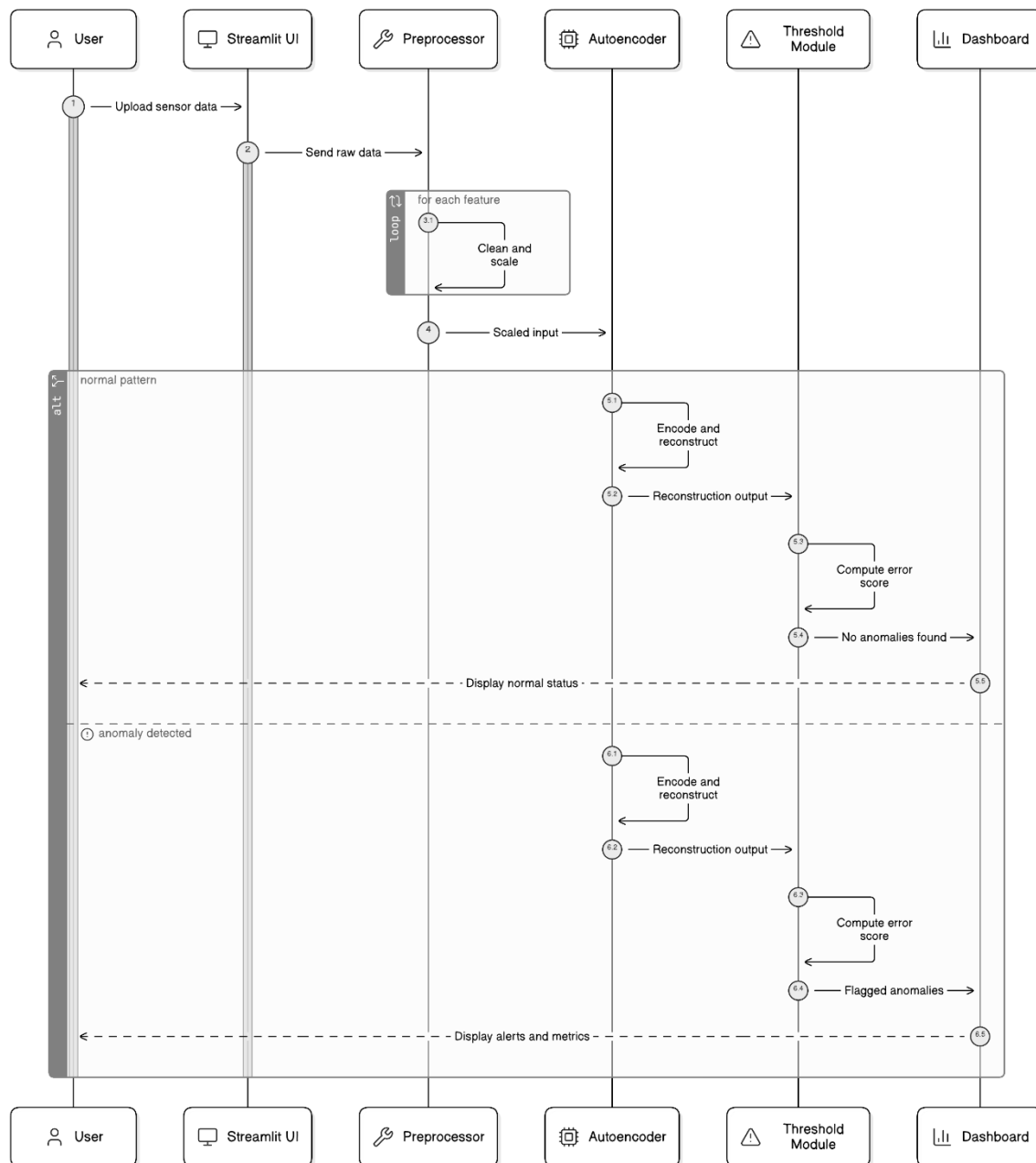


Figure 5.5 above is a UML sequence diagram showing how the message passing sequence works when the administrator uploads a network flow CSV file for the anomaly detection analysis process. The diagram depicts the flow of messages among the five key components of the system: the Streamlit web application UI, the pre-processing component, the autoencoder neural network model, the threshold classification module and the results dashboard component. When the administrator uploads the file using Streamlit interface, the interface passes the uploaded file to the preprocessing component. The preprocessing component then conducts feature cleaning and normalisation via the trained StandardScaler

model. Once normalisation is done, the cleaned matrix is sent to the autoencoder for processing. The autoencoder processes the matrix and produces a reconstructed matrix and the threshold component calculates the MSE score of the row based on the reconstructed matrix. Any row whose MSE score exceeds the calibrated threshold T is labelled as anomalous. Finally, the classification result is sent back to the results dashboard. The proposed interaction model follows a linear flow without any feedback from one component to another. This way, each component in the system is responsible for one task without any circular dependences, making it easy to evaluate each component independently during the testing phase and future maintenance phases.

Section 6. Solution implementation

The system was prototyped as an IoT network anomaly detection system as a streamlit web application. Streamlit is an ideal tool as it helps build interactive data apps written in Python that presents metrics, tables, visualizations etc. The system was prototyped according to the proposed design and compromised five major stages namely, data preprocessing, feature scaling, autoencoder training, reconstruction error scoring and visualization in the dashboard. The implementation was made easier by breaking down the system into components that could independently be evaluated before implementing them as the final pipeline [35].

First data was loaded into Python and examined for missing records and invalid fields. Due to the requirements for autoencoder numerical input, non-numerical identifiers and other irrelevant fields were not included as model input. Data cleaning and scaling was then performed on network flow data features using the StandardScaler function. Data had to be scaled since some features in the network flow data could have vastly different value ranges. Large – value records could overwhelm autoencoder learning process and affect the computation of reconstruction error.

Model training used deep autoencoders that encoded input data into latent space and decode it using a neural network. TensorFlow/Keras libraries include an example of autoencoder models using encoders and decoders [36].

The goal of training the autoencoder was minimization of reconstruction error. Records whose reconstruction error was greater than the threshold were considered anomalies, while those below the threshold were considered normal. A higher anomaly threshold meant that few records were labelled as anomalies, but the vice versa was also true.

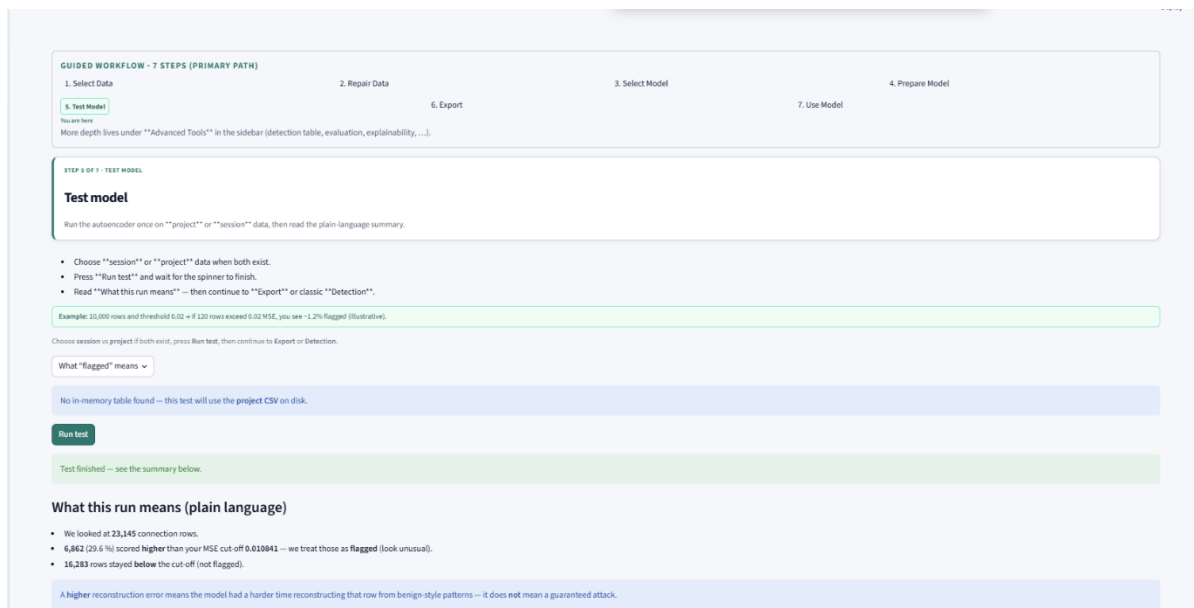
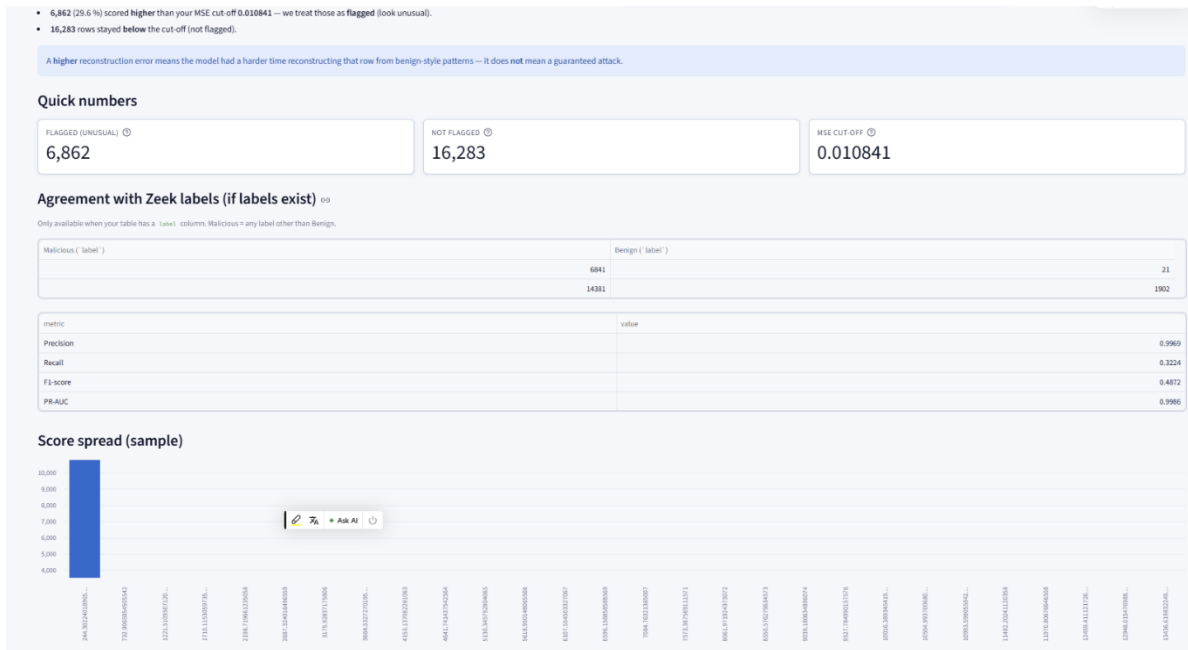
In addition, the model described above an anomaly threshold was defined so that records above the threshold could be labeled as anomalies. Anomalies and records predicted to be normal were classified based on their reconstruction errors; any record with an error greater than the threshold were classified as an anomaly, while one whose reconstruction error was less than or equal to the threshold was classified as normal. This was important for choosing an optimal anomaly threshold that minimized false alarms and attack detection.

The baseline model used an isolation forest algorithm available through Scikit-Learn as an anomaly detection algorithm. This model compared with the autoencoder model. Finally, the model predictions were shown on the Streamlit dashboard alongside various metrics for easy understanding of the model prediction results even by people with little to no knowledge of machine learning [37].

Here is a small table showcasing the main elements of my project as I have a lot of files in my folder.

File/folder	Purpose
App.py	Runs the dashboard
Train autoencoder	Trains the deep autoencoder
Preprocess.py	Cleans and scales dataset features
Evaluate.py	Generates metric and comparison results
Models/	Stores trained model/scaler files
Data/	Contains dataset files
Outputs/	Stores graphs, predictions and results
Requirements.txt	Includes what needs to be used to run

Preprocess the output.



The above results prove the data was well pre-processed before using it to train the model. Preprocessing is necessary because the autoencoder uses only numeric inputs. The preprocessing process helps avoid possible errors due to non-numeric entries, empty cells or data inconsistency issues.

Keras model summary

Model summary (Keras text)

Model: "iot_tabular_autoencoder_v2"

Layer (type)	Output Shape	Param #
features (InputLayer)	(None, 569)	0
enc_1 (Dense)	(None, 128)	72,832
bn_1 (BatchNormalization)	(None, 128)	512
act_1 (Activation)	(None, 128)	0
drop_1 (Dropout)	(None, 128)	0
enc_2 (Dense)	(None, 64)	8,192
bn_2 (BatchNormalization)	(None, 64)	256
act_2 (Activation)	(None, 64)	0
drop_2 (Dropout)	(None, 64)	0
enc_3 (Dense)	(None, 32)	2,048
bn_3 (BatchNormalization)	(None, 32)	128
act_3 (Activation)	(None, 32)	0
bottleneck (Dense)	(None, 16)	528
dec_1 (Dense)	(None, 32)	544
dec_2 (Dense)	(None, 64)	2,112
dec_3 (Dense)	(None, 128)	8,320
reconstruction (Dense)	(None, 569)	73,401

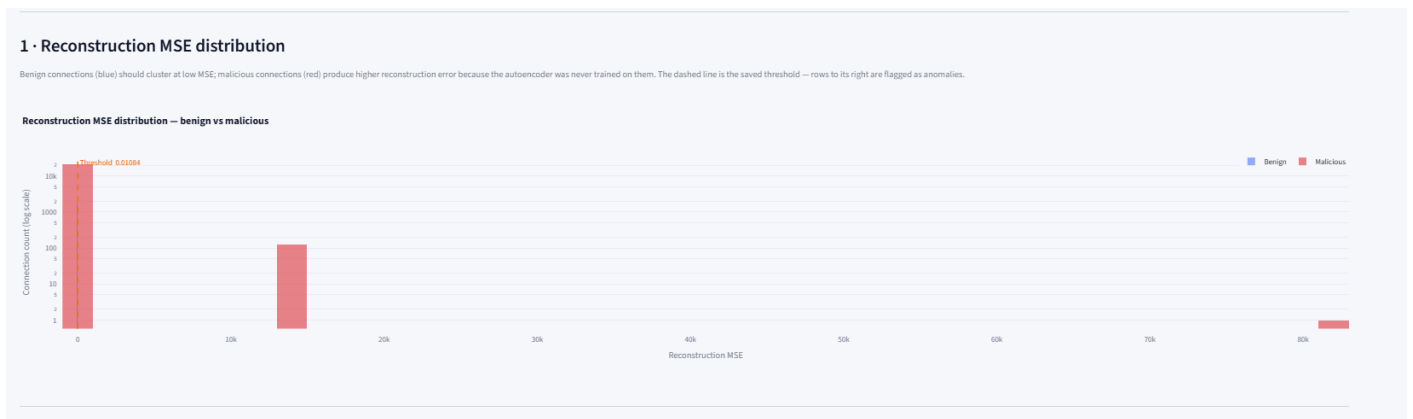
Total params: 168,873 (659.66 KB)

Trainable params: 168,425 (657.91 KB)

Non-trainable params: 448 (1.75 KB)

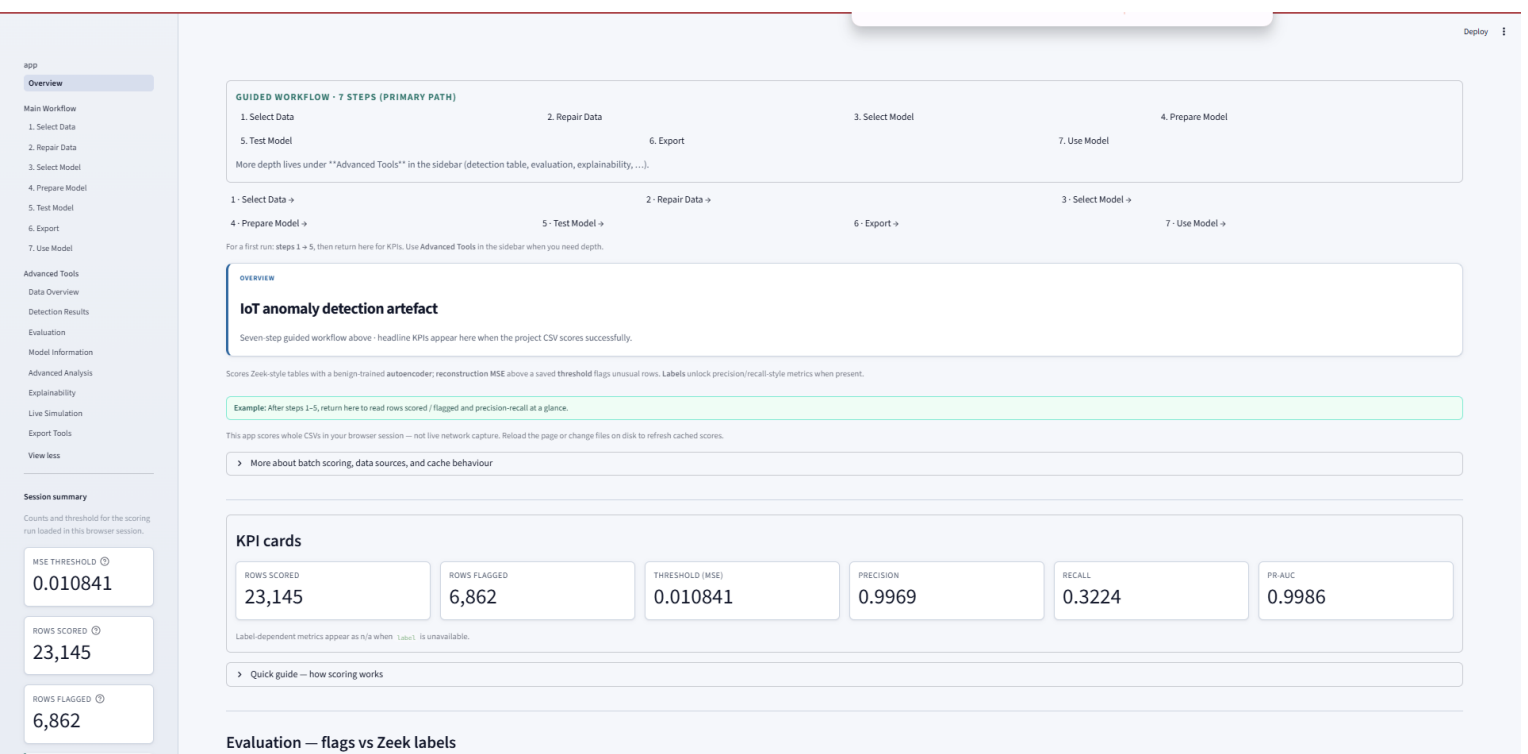
From the model summary provided, one can tell how the deep autoencoder is structured for anomaly detection purposes. The deep autoencoder has several layers that can encode and decode data to reduce it to a more compressed version while being able to detect anomalies using the reconstruction error.

Reconstruction error threshold graph



This chart explains how the concept of reconstruction error was utilized to distinguish normal network traffic from the abnormal one. The line of threshold signifies that any data with a reconstruction error higher than the threshold can be considered an anomaly. It works since any kind of malicious activity would pose a greater challenge to the autoencoder's accurate reconstruction.

Streamlit Dashboard



Streamlit allows creating an interactive dashboard for analysing results of anomaly detection along with important metrics.

Section 7. Solution testing/ evaluation

The system was evaluated through functional tests and model evaluation. Functional tests ensured that the core components performed optimally; these included dataset loading, preprocessing, feature scaling, training of the autoencoder, calculating reconstruction errors, threshold-based classification, prediction by Isolation Forests and dashboard generation.

Assess area	Expected result	Status
Dataset loading	Dataset loads without errors	Fail
Preprocessing	Clean numerical feature matrix is produced	Pass
Model training	Autoencoder trains and loss decreases	Pass
Thresholding	Records are labelled or anomalous	Pass
Dashboard	Metrics, charts and tables are displayed	Pass

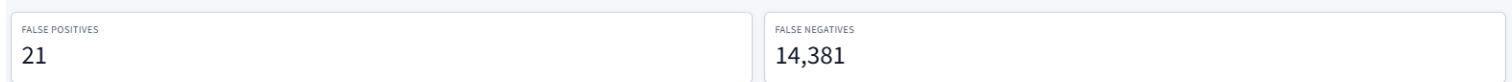
The model performance was evaluated using precision, recall, F1-score and prc-auc. Accuracy was not considered for the main criterion for evaluating the model since the anomaly dataset could be unbalanced. Is it possible to have an accurate model that predicts most instances to be normal while ignoring crucial attacks.

Confusion matrix

Confusion matrix



False positive / false negative analysis



Useful for discussing operational trade-offs at the current threshold.

The confusion matrix is used to compare the prediction made by the algorithm against the actual labels from the labeled dataset. It helps to see the impact of false positives and false negatives on the performance of the algorithm, as in the case of intrusion detection, the consequences of missing an attack are greater than flagging normal data.

Precision recall curve

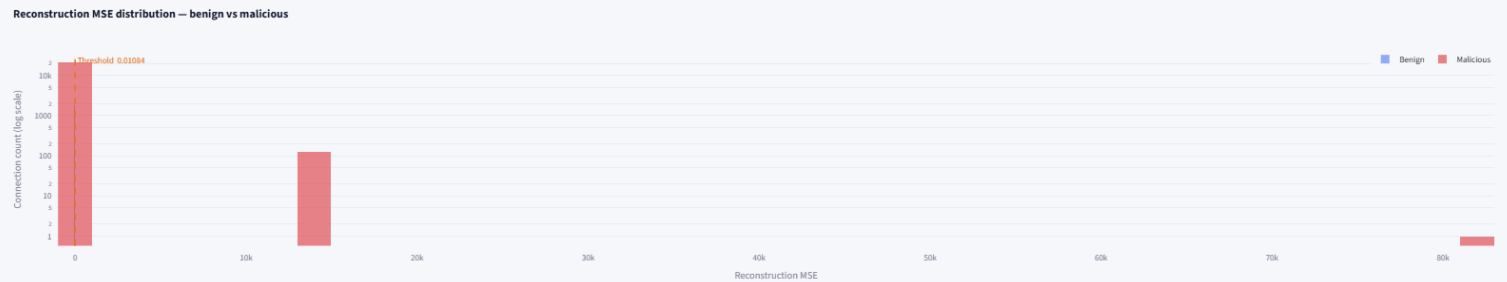


The precision recall graph illustrates the correlation between the two terms across different thresholds. This is important when dealing with an imbalance in the data, where some normal data and others are anomalies to detect these anomalies. The more accurate the graph, the better it suggests that the model works in terms of precision and identifying anomalies.

Reconstruction error distribution

1 · Reconstruction MSE distribution

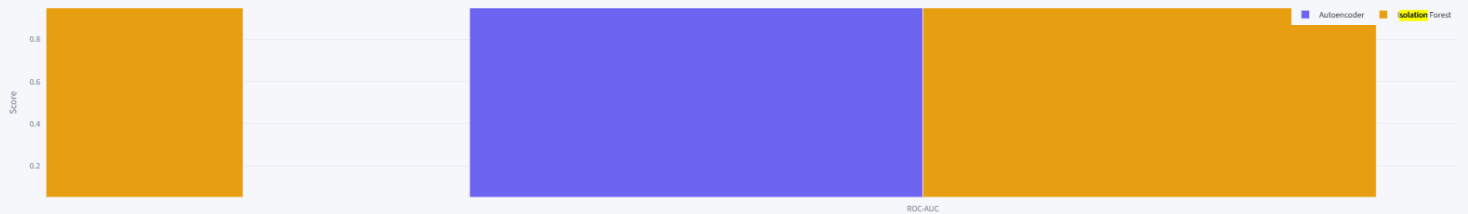
Benign connections (blue) should cluster at low MSE; malicious connections (red) produce higher reconstruction error because the autoencoder was never trained on them. The dashed line is the saved threshold — rows to its right are flagged as anomalies.



The distribution of the reconstruction errors will reveal the distribution of benign and malicious records according to the reconstruction errors. It is expected that the benign data will form clusters for low reconstruction errors, whereas malicious data is seen in high reconstruction errors.

Autoencoder vs Isolation Forest

Autoencoder vs Isolation Forest — key metrics



Autoencoder (current session)

PRECISION
0.9969

RECALL
0.3224

F1
0.4872

PR-AUC
0.9986

ROC-AUC
0.9900

Isolation Forest (test set)

PRECISION
0.0000

RECALL
0.0000

F1
0.0000

PR-AUC
0.9901

ROC-AUC
0.9550

Autoencoder is evaluated against the Isolation Forest model to determine whether the suggested deep learning techniques outperform the anomaly detection system. These findings suggest that the Autoencoder had greater precision, F1-score and PR-AUC.

Section 8. Conclusion

In conclusion, this project has allowed me to gain knowledge in such areas as machine learning, cybersecurity issues and software development. It taught me that creating an effective anomaly detection system is not only about choosing an appropriate algorithm but also involves other tasks related to data preparation, evaluation techniques and user interface design.

Discussion

This project developed, designed and evaluated an interactive anomaly detection system for IoT networks based on deep autoencoder. It aimed at solving the problem of identifying suspicious IoT traffic where labels are scarce and unknown attacks do not follow any signatures. This prototype included preprocessing, feature scaling, reconstruction scoring, threshold-based classification, baselining and a graphical dashboard.

The project achieved its goal by developing a functional prototype of an anomaly detection tool. Results showed that deep autoencoders successfully outperformed in terms of precision being 0.9969 and isolation forest being 0.0000. Recall for the autoencoder is 0.3224 and the isolation forest is 0.0000. The F1 score for autoencoder is 0.4872 and for Isolation Forest is 0.0000. They prove that while reconstruction-based methods can be used effectively, their performance is extremely sensitive to preprocessing and threshold tuning and evaluation metrics used.

However, several limitations have been encountered in this project, namely use of prepared data instead of real time IoT data, fixed threshold value and lack of full explanations provided by the visualisation tools. For future improvements, more advanced evaluation can be done using live data from IoT network, variable thresholding, more complex models compared and further development of explainability approaches.

Section 9. References

- [1] M. Antonakakis *et al.*, “Understanding the Mirai Botnet Open access to the Proceedings of the 26th USENIX Security Symposium is sponsored by USENIX Understanding the Mirai Botnet,” Usenix, 2017. Available: <https://www.usenix.org/system/files/conference/usenixsecurity17/sec17-antonakakis.pdf>
- [2] Y. Meidan *et al.*, “N-BaloT—Network-Based Detection of IoT Botnet Attacks Using Deep Autoencoders,” *IEEE Pervasive Computing*, vol. 17, no. 3, pp. 12–22, Jul. 2018, doi: <https://doi.org/10.1109/mprv.2018.03367731>.
- [3] A. Chatterjee and B. S. Ahmed, “IoT anomaly detection methods and applications: A survey,” *Internet of Things*, vol. 19, p. 100568, Aug. 2022, doi: <https://doi.org/10.1016/j.iot.2022.100568>.
- [4] K. DeMedeiros, A. Hendawi, and M. Alvarez, “A Survey of AI-Based Anomaly Detection in IoT and Sensor Networks,” *Sensors*, vol. 23, no. 3, p. 1352, Jan. 2023, doi: <https://doi.org/10.3390/s23031352>.
- [5] Sebastián García, A. Parmisano, and Maria Jose Erquiaga, “IoT-23: A labeled dataset with malicious and benign IoT network traffic,” Jan. 2020, doi: <https://doi.org/10.5281/zenodo.4743746>.
- [6] N. Koroniotis, N. Moustafa, E. Sitnikova, and B. Turnbull, “Towards the development of realistic botnet dataset in the Internet of Things for network forensic analytics: Bot-IoT dataset,” *Future Generation Computer Systems*, vol. 100, pp. 779–796, Nov. 2019, doi: <https://doi.org/10.1016/j.future.2019.05.041>.
- [7] T. Saito and M. Rehmsmeier, “The Precision-Recall Plot Is More Informative than the ROC Plot When Evaluating Binary Classifiers on Imbalanced Datasets,” *PLOS ONE*, vol. 10, no. 3, p. e0118432, Mar. 2015, doi: <https://doi.org/10.1371/journal.pone.0118432>.
- [8] F. T. Liu, K. M. Ting, and Z.-H. Zhou, “Isolation Forest,” *2008 Eighth IEEE International Conference on Data Mining*, Dec. 2008, doi: <https://doi.org/10.1109/icdm.2008.17>.
- [9] B. Schölkopf, J. C. Platt, J. Shawe-Taylor, A. J. Smola, and R. C. Williamson, “Estimating the Support of a High-Dimensional Distribution,” *Neural Computation*, vol. 13, no. 7, pp. 1443–1471, Jul. 2001, doi: <https://doi.org/10.1162/089976601750264965>.
- [10] M. A. Ferrag, L. Maglaras, S. Moschogiannis, and H. Janicke, “Deep learning for cyber security intrusion detection: Approaches, datasets, and comparative study,” *Journal of Information Security and Applications*, vol. 50, p. 102419, Feb. 2020, doi: <https://doi.org/10.1016/j.jisa.2019.102419>.
- [11] Md Mainuddin, Z. Duan, and Y. Dong, “Detecting Compromised IoT Devices Using Autoencoders with Sequential Hypothesis Testing,” *2021 IEEE International Conference on Big Data (Big Data)*, pp. 1344–1351, Dec. 2023, doi: <https://doi.org/10.1109/bigdata59044.2023.10386399>.
- [12] J. Li, K. Malialis, and M. M. Polycarpou, “Autoencoder-based Anomaly Detection in Streaming Data with Incremental Learning and Concept Drift Adaptation,” *2023 International Joint Conference on Neural Networks (IJCNN)*, pp. 1–8, Jun. 2023, doi: <https://doi.org/10.1109/ijcnn54540.2023.10191328>.

- [13] H. Rhachi, Y. Balboul, and A. Bouayad, "Enhanced Anomaly Detection in IoT Networks Using Deep Autoencoders with Feature Selection Techniques," *Sensors*, vol. 25, no. 10, p. 3150, May 2025, doi: <https://doi.org/10.3390/s25103150>.
- [14] W. Xu and Y. Fan, "Intrusion Detection Systems Based on Logarithmic Autoencoder and XGBoost," *Security and Communication Networks*, vol. 2022, pp. 1–8, Apr. 2022, doi: <https://doi.org/10.1155/2022/9068724>.
- [15] "TinyML [Book]," www.oreilly.com.
<https://www.oreilly.com/library/view/tinyml/9781492052036/>
- [16] A. Singh and J. Jang-Jaccard, "Autoencoder-based Unsupervised Intrusion Detection using Multi-Scale Convolutional Recurrent Networks," *arXiv.org*, Apr. 07, 2022. <https://arxiv.org/abs/2204.03779>
- [17] M. E. Mahmoud, M. Kasem, A. Abdallah, and H.-K. Kang, "AE-LSTM: Autoencoder with LSTM-Based Intrusion Detection in IoT," *2022 International Telecommunications Conference (ITC-Egypt)*, Jul. 2022, doi: <https://doi.org/10.1109/itc-egypt55520.2022.9855688>.
- [18] S. Givnan, C. Chalmers, P. Fergus, S. Ortega, and T. Whalley, "Real-Time Predictive Maintenance using Autoencoder Reconstruction and Anomaly Detection," *arXiv.org*, Oct. 01, 2021.
<https://arxiv.org/abs/2110.01447> (accessed Feb. 12, 2024).
- [19] A. A. Nelay and M. Turgeon, "A Comprehensive Study of Auto-Encoders for Anomaly Detection: Efficiency and Trade-Offs," 2024, doi: <https://doi.org/10.2139/ssrn.4757204>.
- [20] S. Sinha, "State of IoT 2024: Number of connected IoT devices growing 13% to 18.8 billion globally," *IoT Analytics*, Sep. 03, 2024. <https://iot-analytics.com/number-connected-iot-devices/>
- [21] F. Maleki, K. Ovens, R. Gupta, C. Reinhold, A. Spatz, and R. Forghani, "Generalizability of Machine Learning Models: Quantitative Evaluation of Three Methodological Pitfalls," *Radiology: Artificial Intelligence*, vol. 5, no. 1, Nov. 2022, doi: <https://doi.org/10.1148/ryai.220028>.
- [22] M. B. Musthafa *et al.*, "Optimizing IoT Intrusion Detection Using Balanced Class Distribution, Feature Selection, and Ensemble Machine Learning Techniques," *Sensors*, vol. 24, no. 13, p. 4293, Jul. 2024, doi: <https://doi.org/10.3390/s24134293>.
- [23] S. More, Moad Idrissi, H. Mahmoud, and A. Taufiq Asyhari, "Enhanced Intrusion Detection Systems Performance with UNSW-NB15 Data Analysis," *Algorithms*, vol. 17, no. 2, pp. 64–64, Feb. 2024, doi: <https://doi.org/10.3390/a17020064>.
- [24] S. Beddar-Wiesing, A. Moallem-Oureh, M. Kempkes, and J. M. Thomas, "Absolute Evaluation Measures for Machine Learning: A Survey," *arXiv.org*, 2025. <https://arxiv.org/abs/2507.03392>
- [25] L. Romao, H. Villamizar, R. Oliveira, S. Alonso, and M. Kalinowski, "Agile Management for Machine Learning: A Systematic Mapping Study," *arXiv (Cornell University)*, Jun. 2025, doi: <https://doi.org/10.48550/arxiv.2506.20759>.
- [26] "Model training and tuning - Machine Learning Lens," *Amazon.com*, 2024.
<https://docs.aws.amazon.com/wellarchitected/latest/machine-learning-lens/model-training-and-tuning.html>

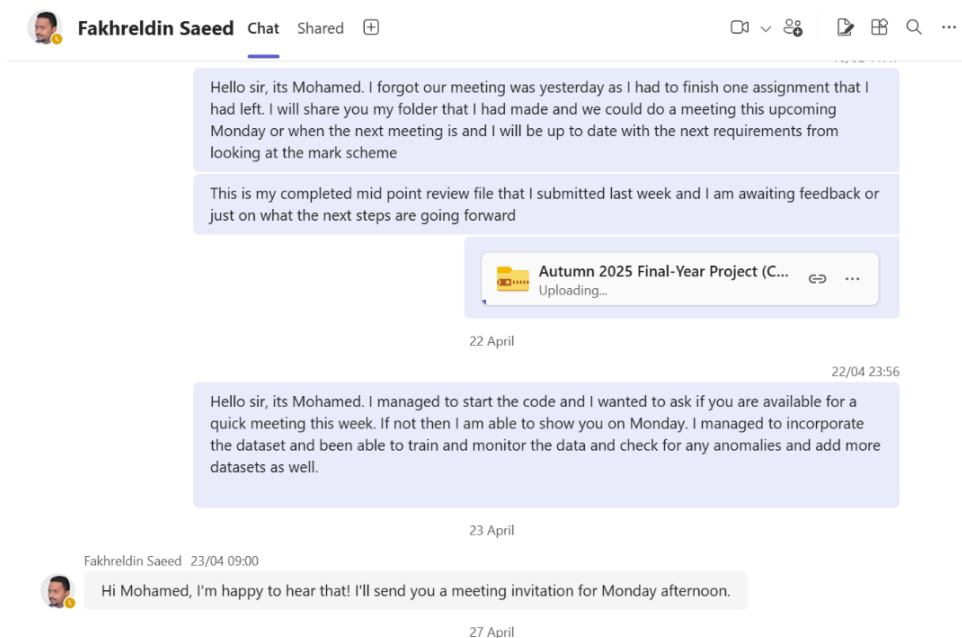
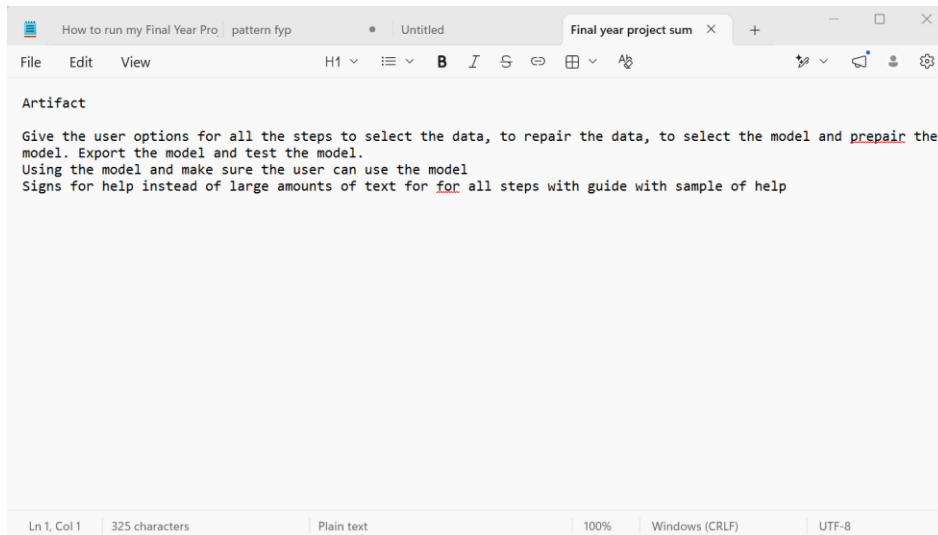
- [27] E. Kavlakoglu, “Agile vs. Waterfall: What’s the Difference? | IBM,” *www.ibm.com*, Jul. 25, 2024. <https://www.ibm.com/think/topics/agile-vs-waterfall>
- [28] D. Radigan, “Agile vs. waterfall project management,” *Atlassian*. <https://www.atlassian.com/agile/project-management/project-management-intro>
- [29] T. Runcie, “Risk Management Agile v Waterfall (Runcie),” *pmi-portland.org*. <https://pmi-portland.org/news-and-content/675-risk-management-agile-v-waterfall>
- [30] J. Jordan, “Hyperparameter tuning for machine learning models,” *Jeremy Jordan*, Nov. 02, 2017. <https://www.jeremyjordan.me/hyperparameter-tuning/>
- [31] “Internet of Things (IoT) and the GDPR – How to stay compliant,” *Bulletproof.co.uk*. <https://www.bulletproof.co.uk/blog/iot-and-gdpr-how-to-stay-compliant>
- [32] C. W. Johnson, “Barriers to the Use of Intrusion Detection Systems in Safety-Critical Applications,” *Lecture Notes in Computer Science*, pp. 375–384, 2015, doi: https://doi.org/10.1007/978-3-319-24255-2_27.
- [33] Association for Computing Machinery, “ACM Code of Ethics and Professional Conduct,” *Association for Computing Machinery*, Jun. 22, 2018. <https://www.acm.org/code-of-ethics>
- [34] BCS, “BCS Code of Conduct | BCS,” *www.bcs.org*, 2022. <https://www.bcs.org/membership-and-registrations/become-a-member/bcs-code-of-conduct/>
- [35] Streamlit, “Streamlit Docs,” *docs.streamlit.io*, 2024. <https://docs.streamlit.io/>
- [36] “Intro to Autoencoders | TensorFlow Core,” *TensorFlow*. <https://www.tensorflow.org/tutorials/generative/autoencoder>
- [37] Scikit Learn, “sklearn. ensemble. IsolationForest — scikit-learn 0.23.1 documentation,” *scikit-learn.org*. <https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.IsolationForest.html>
- [38] scikit-learn, “StandardScaler,” *scikit-learn.org*, 2019. <https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.StandardScaler.html>
- [39] Stratosphere IPS, “Stratosphere Laboratory,” *Stratosphere Laboratory*, Jan. 22, 2020. <https://www.stratosphereips.org/blog/2020/1/22/aposemat-iot-23-a-labeled-dataset-with-malicious-and-benign-iot-network-traffic> (accessed May 11, 2026).
- [40] “Aposemat IoT-23,” *Cvut.cz*, 2026. <https://cs.fel.cvut.cz/en/news/detail/1536> (accessed May 11, 2026).
- [41] M. I. Ghafoor, Muhammad Sohaib Roomi, M. S. Khan, Mehmood Baryalai, S. Hussain, and I. Tabassum, “Enhancing IoT Cybersecurity through Multi-Technique Data Anonymization: A Differential Privacy Framework Using Public IoT Datasets,” *The Asian Bulletin of Big Data Management*, vol. 6, no. 1, pp. 281–292, Mar. 2026, doi: <https://doi.org/10.62019/tdjz6f05>.

Appendices

GitHub link:

<https://github.com/ibrahimm2106/iot-autoencoder-artifact>

Supervisor meetings / notes



Useability test link

https://docs.google.com/forms/d/e/1FAIpQLSfQNvwbCIH7U4_PkTF70ny7dpJdHHzTRHvmH0O1leTt-DF4w/viewform



Name: Sarah Lim

Demographics

Gender: Sarah Lim

Age: 26

Location: Suburban apartment

Relationship Status: Not relevant

Title:

System
scheduler

Education: Diploma / Undergraduate

Goals

- Know when a device might be formed.
- Keep home devices private and secure.
- Fix issues quickly with simple steps.

Motivations

- Avoid privacy invasion.
- Avoid devices being used in botnet attacks.
- Peace of mind

Background Description

Sarah uses smart cameras, a smart speaker, and smart plugs at home. She does not understand network security, but she worries about privacy and wants reassurance that devices are not doing something weird. She will not interpret packet logs, and she will not tune complex settings.

Frustrations

- Security tools feel technical and confusing.
- Unclear which devices caused the issue.
- Too many warnings that turn out harmless

Expectations:

- Simple normal / suspicious / elevated risk
- Device level identification
- Minimal configuration needed
- Suggested actions

Earlier Experience:

- Basic router admin page
- No ML or networking monitoring experience
- Has relied on antivirus apps before.



Name: Amir Hassan

Demographics

Gender: Male

Age: 31

Location: UK

Relationship Status: [Input Status]

Title: **SME
Net-
work/Se-
curity
Adminis-
trator**

Education: Bsc IT / Networking
certifications

Goals

- Implement an end-to-end pipeline.
- Achieve defensible results using benchmark datasets.
- Compare autoencoder vs a baseline method fairly.
- Produce a well structured report aligned to

Motivations

- High grade and strong final report
- Build a portfolio quality security /ML project.
- Learn reproducible ML experimentation habits.

Background Description

Daniel is building an anomaly detection prototype for IoT traffic using autoencoders. He needs to meet academic marking criteria and is comfortable in terms of coding and does need structure as well as datasets that need consistent preprocessing and with experiment tracking and clear documentation.

Frustrations

- Datasets have inconsistent formats and labels.
- Preprocessing choices can silently change results.
- Time constraints and scope creep
- Difficulty explaining model decisions clearly in the report.

Expectations:

- Clear experimental design
- Logged hyperparameters, results, and plots.
- Reasonable runtime on a student machine
- Traceable outputs in the report

Earlier Experience:

- Familiar with Python ML libraries and basic neural nets
- Has done supervised ML coursework.
- Limited real world security operations exposure